



Bao Cao Do An CSNM - Đồ án cơ sở ngành mạng tìm hiểu về giao tiếp đường ống trong hệ điều hành và

Mạng Máy Tính (Trường Đại học Bách Khoa - Đại học Đà Nẵng)



Scan to open on Studeersnel



**TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CÔNG NGHỆ THÔNG TIN**



BÁO CÁO

ĐỒ ÁN CƠ SỞ NGÀNH MẠNG

ĐỀ TÀI:

- 1. CHƯƠNG TRÌNH TÍNH TOÁN CÁC PHÉP TÍNH
SỬ DỤNG GIAO TIẾP ĐƯỜNG ỐNG**
- 2. XÂY DỰNG ỨNG DỤNG CẬP NHẬT TIN TỨC
THÔNG QUA RSS FEED**

Sinh viên: Lê Trí Tâm
Giáo viên hướng dẫn: TS. Trần Lê Đức

Đà Nẵng ngày 22 tháng 11 năm 2022

MỤC LỤC

PHẦN 1: HỆ ĐIỀU HÀNH	4
I. Cơ sở lý thuyết:	4
a. Hệ điều hành:	4
b. Tiến trình và việc giao tiếp giữa các tiến trình:	5
c. Giao tiếp giữa các tiến trình thông qua đường ống (pipe):	5
II. Đề tài và mô tả bài toán:	6
III. Các thuật toán:	7
a. Thuật toán đồng bộ:	7
b. Thuật toán xử lý dữ liệu:	10
c. Thuật toán xử lý các biệt lệ:	13
d. Thuật toán chi tiết tiến trình cha:	17
e. Thuật toán chi tiết tiến trình con:	18
IV. Demo chương trình:	20
a. Mã nguồn tổng quan chương trình:	20
b. Mã nguồn tiến trình cha:	24
c. Mã nguồn tiến trình con:	25
d. Kết quả chạy chương trình:	30
PHẦN 2: QUẢN TRỊ MẠNG	32
I. Cơ sở lý thuyết:	32
a. Mô hình Client-Server:	32
b. Mô hình truyền tin Socket:	33
c. Socket.IO:	33
d. Sơ lược về RSS:	35
II. Mô tả đề tài:	37
III. Thuật toán chương trình:	37
a. Youtube Search API:	37
b. Youtube RSS Feed Extension:	39
c. Thuật toán chuyển đổi XML thành JSON:	40
IV. Mã nguồn chương trình và demo:	44
a. Mã nguồn Client:	44
b. Mã nguồn Server:	47
c. Hình ảnh demo:	50
PHỤ LỤC VÀ TÀI LIỆU THAM KHẢO	54

DANH MỤC HÌNH ẢNH

Hình 1: Biến hóa giao diện xấu xí của phần cứng trở nên xinh đẹp.....	4
Hình 2. Giao tiếp giữa tiến trình P1 và P2 thông qua pipe.....	6
Hình 3. Cấu trúc tập tin đọc.....	6
Hình 4. Hai tiến trình giao tiếp bằng hai đường ống.....	7
Hình 5. Mô hình giao tiếp.....	8
Hình 6. Sơ đồ thuật toán đồng bộ hai tiến trình.....	9
Hình 7. Thuật toán xử lý dữ liệu của tiến trình cha.....	10
Hình 8. Hàm gộp các thành phần của mảng số thực thành một số thực.....	11
Hình 9. Thuật toán xử lý dữ liệu của tiến trình con.....	12
Hình 10. Thuật toán xử lý biệt lệ tiến trình cha.....	14
Hình 11. Hàm exeOP() xử lý biệt lệ khi ký tự nhận được là một toán tử.....	15
Hình 12. Thuật toán xử lý biệt lệ tiến trình con.....	16
Hình 13. Thuật toán tiến trình cha.....	17
Hình 14. Hàm exeOP() xử lý biệt lệ cho ký tự toán tử.....	18
Hình 15. Thuật toán chi tiết tiến trình con.....	19
Hình 16. Thông tin phép tính.....	30
Hình 17. Kết quả tính toán.....	31
Hình 18. Kết quả chạy và thông tin xuất ra console.....	31
Hình 19. Client gửi yêu cầu và Server phản hồi.....	32
Hình 20. Socket là những cái ổ cắm điện.....	33
Hình 21. Giao tiếp hai chiều giữa Client và Server.....	34
Hình 22. Tổng quan việc kết nối với Socket.IO.....	35
Hình 23. Logo của RSS.....	36
Hình 24. Giao diện của Feedly, một ứng dụng đọc tin RSS.....	36
Hình 25. Thuật toán đệ quy chuyển đổi XML thành Javascript Object.....	42
Hình 26. Giao diện tổng quan trang web.....	50
Hình 27. Chức năng thêm kênh mới.....	50
Hình 28. Tìm kiếm và thêm kênh Ted-Ed.....	51
Hình 29. Giao diện thông tin kênh mới thêm vào.....	51
Hình 30. Tiến hành xóa kênh.....	52
Hình 31. Giao diện sau khi xóa.....	52
Hình 32. Nội dung console của Server khi thao tác các bước trên client.....	53

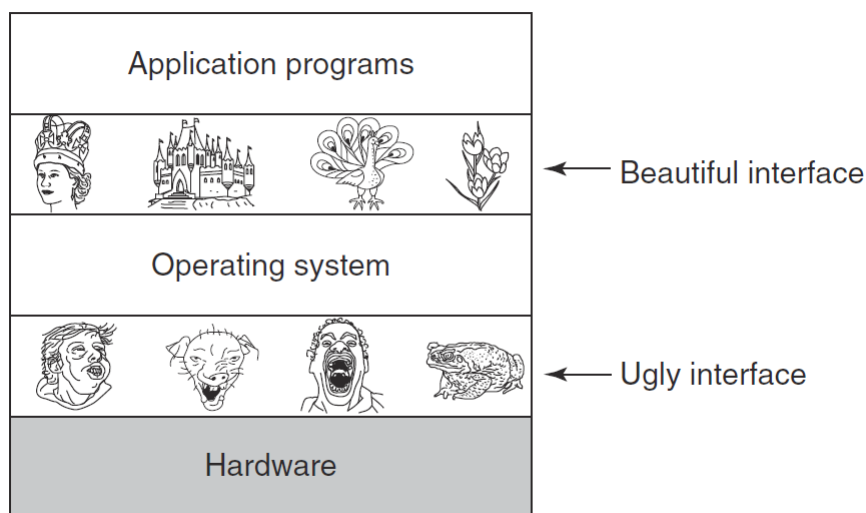
PHẦN 1: HỆ ĐIỀU HÀNH

I. Cơ sở lý thuyết:

a. Hệ điều hành:

Rất khó để cắt nghĩa hoàn hảo một hệ điều hành là gì. Có thể hiểu hệ điều hành là một chương trình chạy ở chế độ nhân (kernel) và chịu trách nhiệm thực hiện hai chức năng: một là cung cấp các chương trình ứng dụng cho người dùng - sự trừu tượng hóa các tài nguyên phần cứng, và hai là quản lý tổ chức các tài nguyên phần cứng đó trên máy tính.

Với chức năng đầu tiên, hệ điều hành giống như một cỗ máy mở rộng. Hệ điều hành cung cấp một lớp giao diện và một phương pháp giao tiếp dễ tiếp cận hơn để người dùng có thể tương tác với máy mà không cần hiểu những phần cứng ẩn bên dưới hoạt động như thế nào.



Hình 1: Biến hóa giao diện xấu xí của phần cứng trở nên xinh đẹp

Song song với việc cung cấp những chương trình ứng dụng có giao diện và phương pháp tương tác “đẹp” hơn cho người dùng. Hệ điều hành còn giúp quản lý các tài nguyên mà chính những chương trình ứng dụng đó cần được cung cấp. Tài nguyên nào được sử dụng và chương trình nào được phép chạy và chạy trong khoảng thời gian bao lâu sẽ được hệ điều hành quản lý.

Một số loại hệ điều hành phổ biến cho người dùng phổ thông như Windows 10, Windows 11 và MacOS. Trong báo cáo này, các chương trình sẽ được lập trình để

chạy trên Ubuntu và Windows 11.

b. Tiến trình và việc giao tiếp giữa các tiến trình:

Có rất nhiều đầu việc mà máy tính phải xử lý để thực hiện một tác vụ nào đó mà con người yêu cầu. Những việc đó được chia nhỏ ra thành các việc nhỏ hơn và nhỏ hơn nữa. Và một đơn vị công việc xử lý đó ta gọi là một **tiến trình**.

Tiến trình có thể hiểu là đơn vị công việc nhỏ nhất mà máy tính cần xử lý. Một chương trình lớn sẽ được cấu thành từ rất nhiều tiến trình. Ở quy mô các chương trình ứng dụng của người dùng thì máy tính là một thiết bị đa nhiệm nhưng ở quy mô vi mô của tiến trình thì máy tính chỉ hoạt động đơn nhiệm, nghĩa là trong một thời điểm bất kỳ chỉ có duy nhất một tiến trình được thực hiện mà thôi.

Các tiến trình thuộc một chương trình lớn chắc chắn sẽ phải có những sự giao tiếp với nhau. Chúng có thể sử dụng tài nguyên chung với nhau. Kết quả xử lý của tiến trình này có thể là đầu vào của tiến trình kia và ngược lại. Vì thế ngoài việc quản lý thời gian hoạt động cho các tiến trình, tài nguyên nào được cấp phát cho tiến trình thì hệ điều hành cũng phải quản lý việc giao tiếp giữa các tiến trình với nhau.

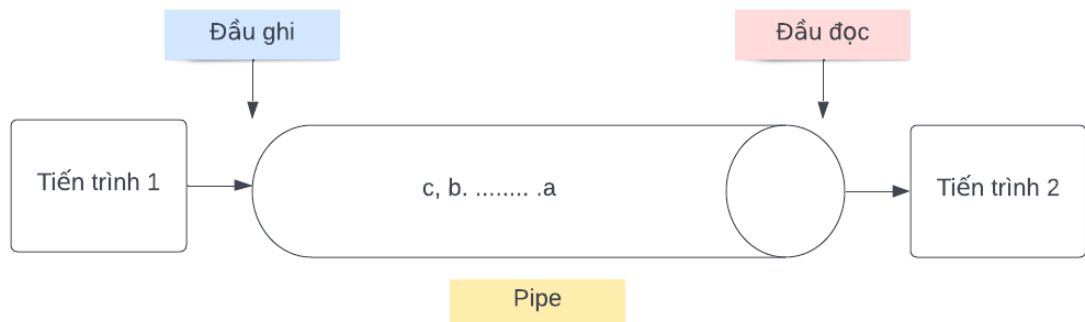
Có những cách thức giao tiếp giữa các tiến trình như sau:

1. Giao tiếp bằng bộ nhớ dùng chung (shared memory)
2. Giao tiếp bằng bộ nhớ ánh xạ (mapped memory)
3. Giao tiếp thông qua đường ống (pipe)

c. Giao tiếp giữa các tiến trình thông qua đường ống (pipe):

Đường ống (hay pipe) là một cách thức giao tiếp tuần tự giữa các tiến trình. Có thể hiểu nó như một đường ống một chiều có một đầu vào và một đầu ra. Một tiến trình nào đó có thể ghi dữ liệu vào ống từ đầu vào và một tiến trình khác có thể đọc dữ liệu đó từ đầu ra.

Đường ống chứa các chuỗi byte, thứ tự của chúng tuân theo quy tắc FIFO, nghĩa là byte nào được ghi vào trước thì sẽ được đọc ra trước. Một pipe như vậy cũng có kích thước cố định. Nếu dữ liệu trong pipe đã đầy từ đầu ghi sẽ bị đóng lại đến khi nào có tiến trình nào đó đọc bớt dữ liệu trong pipe ra.



Hình 2. Giao tiếp giữa tiến trình P1 và P2 thông qua pipe

Có hai loại đường ống pipe:

- **Normal pipe** (đường ống thông thường): Đường ống này bị giới hạn trong bộ nhớ của một tiến trình mà thôi. Nó là đường ống nội bộ của tiến trình đó. Đường ống thông thường chỉ giúp giao tiếp giữa tiến trình nó thuộc về với các tiến trình con của tiến trình đó. Có thể hiểu như đây là một kênh liên lạc nội bộ cha-con của một tiến trình.
- **Named pipe** (đường ống có tên): Là một đối tượng độc lập được có tên trong hệ thống. Nó cho phép các tiến trình có không gian địa chỉ khác nhau có thể giao tiếp với nhau. Chính vì vậy mà nó cần được đặt tên để có thể được kết nối. Thực chất loại đường ống này là một tập tin mà được quy định hai đầu, một đầu ghi và một đầu đọc!

II. Đề tài và mô tả bài toán:

Đề tài của phần báo cáo này là:

Viết chương trình tính toán các phép tính sử dụng giao tiếp đường ống

Bài toán cụ thể của chúng ta sẽ là:

Cho một tập tin có tên là readFile.txt chứa các biểu thức tính toán hai số thực có cấu trúc như sau:

1	-2*3
2	34+67
3	44.6+89.9
4	9.6+7.8

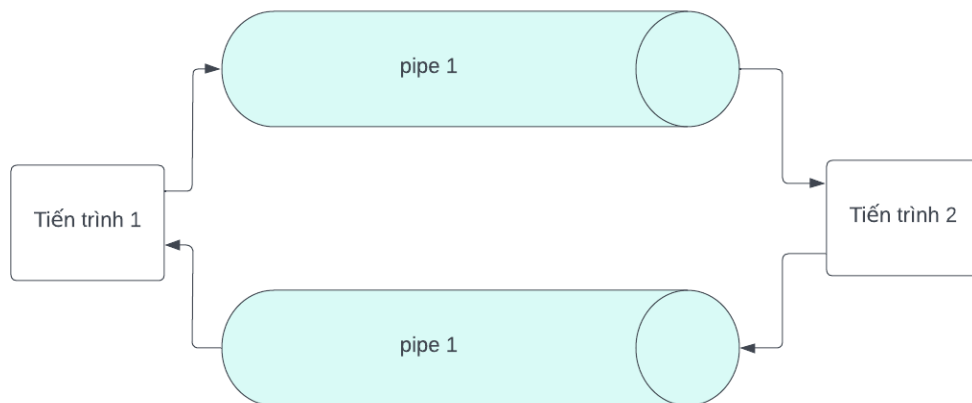
Hình 3. Cấu trúc tập tin đọc

Viết chương trình sử dụng giao tiếp đường ống (pipe) để viết lại vào tập tin

khác các biểu thức đó kèm với kết quả theo sau, chú ý đến các vấn đề về biệt lệ.

Với bài toán trên, ta thấy rằng phải sử dụng đường ống thông thường vì chỉ được viết một chương trình mà thôi. Ta cũng sẽ phải có hai tiến trình, một tiến trình sẽ giao tiếp với tập tin và một tiến trình sẽ tính toán. Ta cũng cần đến hai đường ống để giao tiếp giữa hai tiến trình này. Một đường ống sẽ được dùng để đọc các biểu thức vào tiến trình tính toán và một đường ống sẽ được dùng để lấy kết quả sau khi đã tính toán.

Ta cũng phải chú ý đến những biệt lệ khác như chia cho 0, ký tự nhập vào không giúp cấu thành một số và một số vấn đề khác như ký tự xuống dòng, ký tự kết thúc tập tin (EOF).

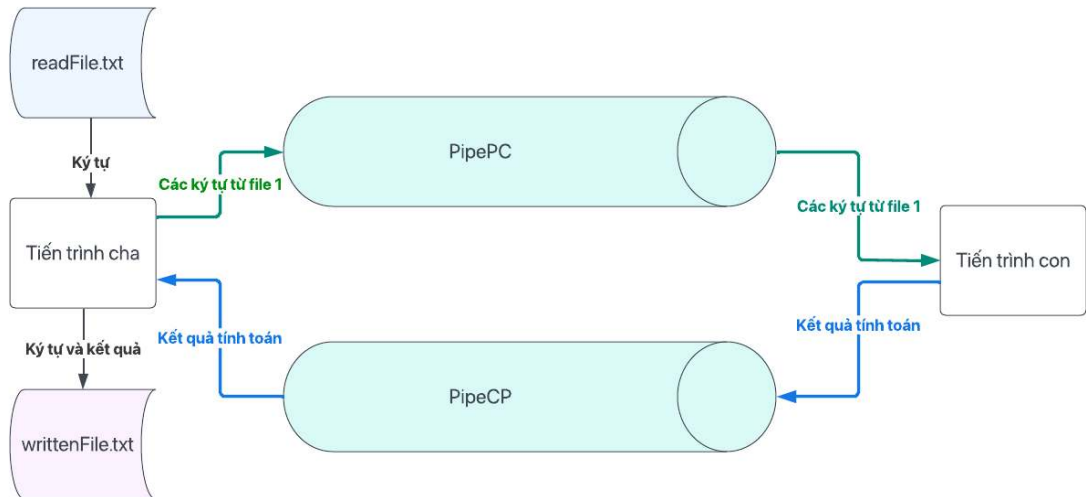


Hình 4. Hai tiến trình giao tiếp bằng hai đường ống

III. Các thuật toán:

a. Thuật toán đồng bộ:

Dựa trên những gì đã phân tích ở phần trước, chúng ta sẽ xây dựng được một mô hình giao tiếp gồm các thành phần như sau:



Hình 5. Mô hình giao tiếp

Ta sẽ có hai tập tin: tập tin đầu tiên chứa các phép tính số thực, tập tin thứ hai sẽ là tập tin đích cần ghi vào. Ta cũng có hai tiến trình là tiến trình cha và tiến trình con.

Tiến trình cha sẽ đảm nhiệm việc giao tiếp với hai file. Tiến trình này đọc các ký tự từ file 1 vào và ghi chúng vào file 2, đồng thời ghi vào pipePC để tiến trình con có thể sử dụng để xử lý. Tiến trình cha sau đó nhận kết quả trả về từ tiến trình con bằng cách đọc kết quả từ pipeCP rồi ghi vào file 2.

Tiến trình con sẽ nhận các phép tính từ tiến trình cha từ pipePC, xử lý chúng sau đó trả lại kết quả vào pipe CP.

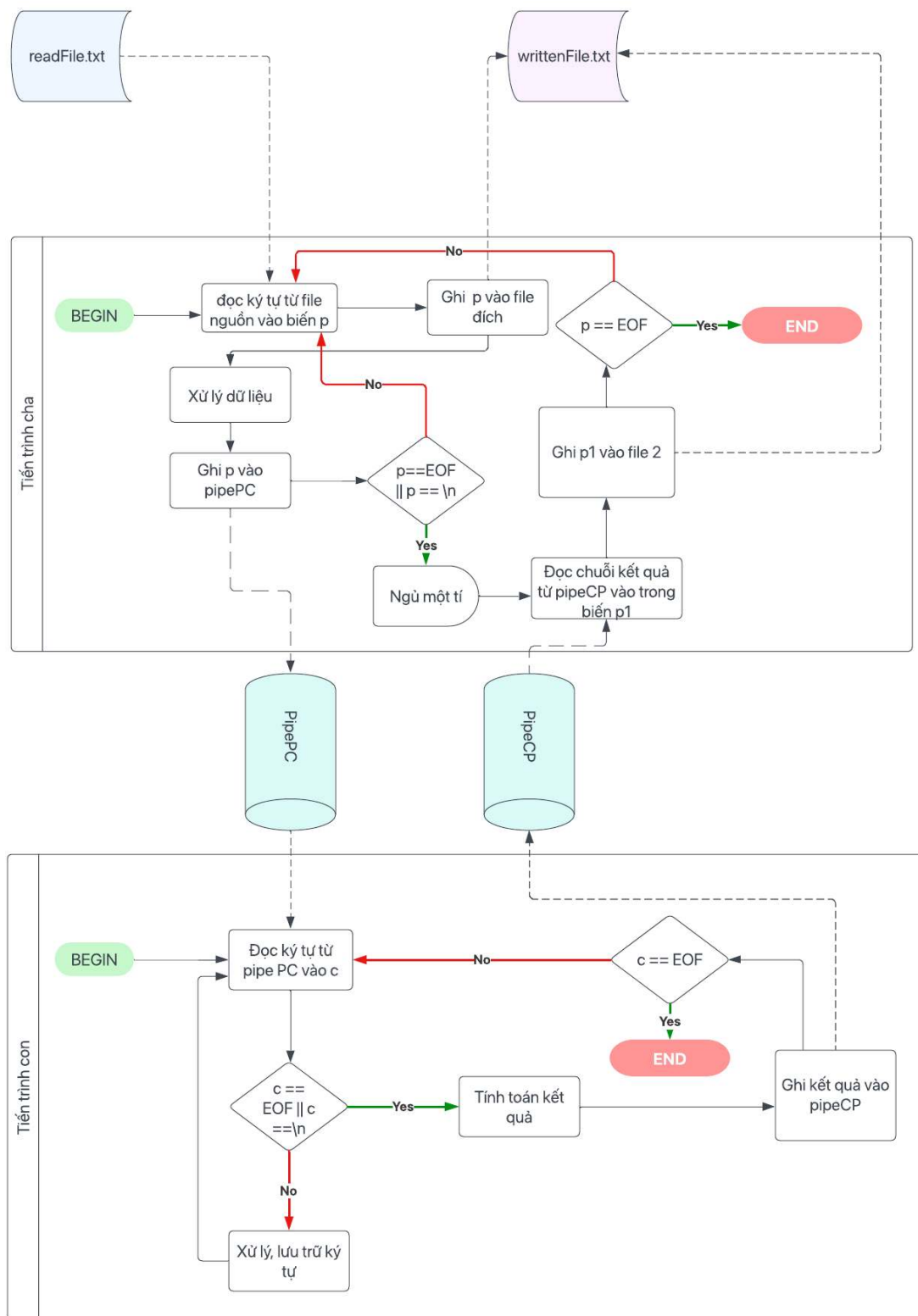
Quy tắc đọc là cả tiến trình cha và tiến trình con sẽ đọc và ghi từng ký tự một. Các phép tính liên nhau và khi ghi vào file 2 thì các phép tính xen kẽ với các kết quả. Vì thế ta cần phải đồng bộ cả quá trình này. Ta tiến hành đồng bộ theo thuật toán như sau:

Tiến trình cha đọc từng ký tự ở file 1, ghi nó vào file 2, xử lý vài bước rồi ghi vào pipePC, nếu đọc/ghi ký tự xuống dòng $\backslash n$ hoặc EOF (end of file) thì ngừng vài giây chờ cho con xử lý phép tính

Tiến trình con nhận từng ký tự từ pipePC và lưu trữ các ký tự vào. Nếu đọc phải ký tự $\backslash n$ hoặc EOF thì thực hiện phép tính và ghi kết quả vào pipeCP. Nếu đọc phải ký tự EOF thì tiến trình con kết thúc.

Tiến trình cha đọc được kết quả từ pipeCP sẽ ghi vào file 2. Nếu trước đó đọc ký tự EOF thì kết thúc. Nếu không thì tiếp tục đọc ghi ký tự từ file 1.

Ta có sơ đồ thuật toán đồng bộ như hình 6.



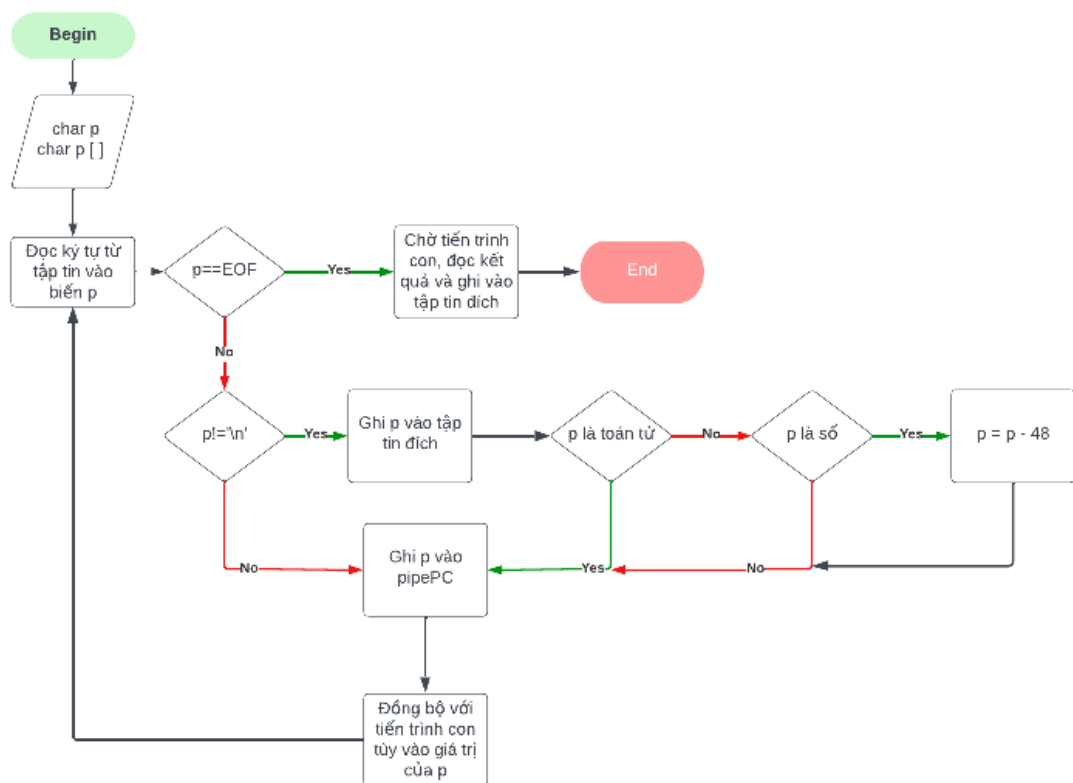
Hình 6. Sơ đồ thuật toán đồng bộ hai tiến trình

Như vậy dựa vào các ký tự $\backslash n$ và EOF thì ta đã có thể đồng bộ hai tiến trình với nhau, đảm bảo tiến trình cha sẽ luôn kết thúc sau tiến trình con và kết quả sẽ luôn

được đọc xen kẽ với các phép tính.

b. Thuật toán xử lý dữ liệu:

Đối với tiến trình cha, nhiệm vụ chính vẫn là giao tiếp với các tập tin và các đường ống. Vì vậy đồng bộ mới là vấn đề quan trọng nhất mà tiến trình cha cần xử lý. Về việc xử lý dữ liệu thì tiến trình cha sẽ chuyển đổi các ký tự chữ số về thành số để tiến trình con tính toán sau đó. Ta có thuật toán xử lý dữ liệu của tiến trình cha như sau:



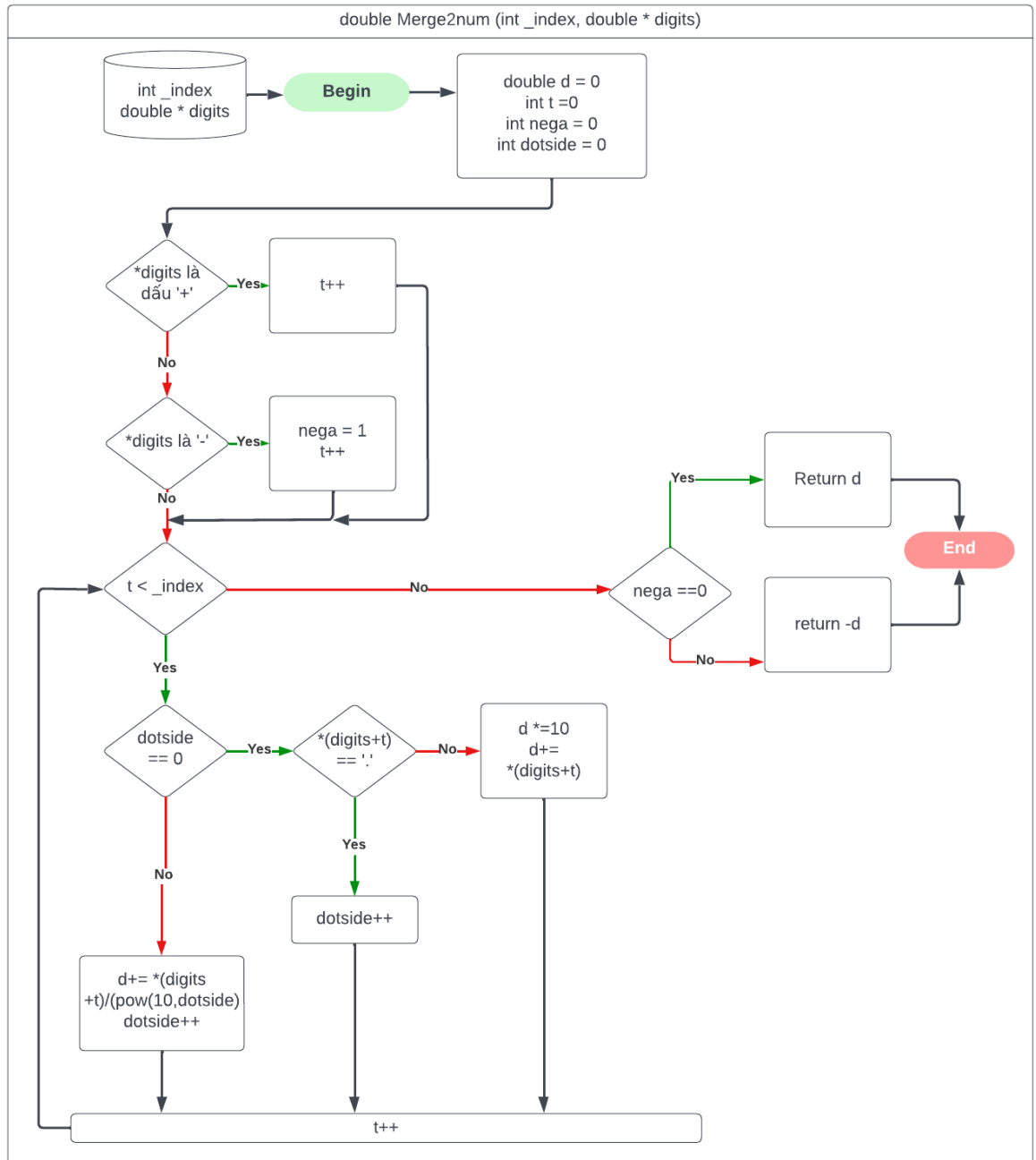
Hình 7. Thuật toán xử lý dữ liệu của tiến trình cha

Đối với tiến trình con, quá trình xử lý sẽ phức tạp hơn khá nhiều khi tiến trình con phải thu thập từng ký tự một, nhận biết đâu là kết thúc của một số hạng và khi nào thì tính toán để ghi trả lại cho tiến trình cha. Ta có thể mô tả thuật toán xử lý dữ liệu của tiến trình con như sau:

- Tiến trình con liên tục đọc ký tự từ pipePC và lưu vào một mảng số thực là *digits[]*.
- Nếu gặp toán tử, tiến trình con sẽ gộp tất cả thành phần của mảng *digits[]* lại thành một số thực và lưu nó vào mảng *nums[]*. Sau đó lưu lại toán tử

- Nếu gặp phải ký tự xuống dòng là `\n` thì tiến trình con cũng gộp tất cả thành phần của `digits[]` lại lưu vào mảng `nums`. Sau đó dựa vào toán tử đã lưu tính toán tương ứng với `nums[index]` và `nums[index-1]`.

Như vậy ta cần một hàm để gộp các chữ số của một mảng số thực lại thành một số thực duy nhất. Ta có thuật toán của hàm `MergeToNum()` như sau:

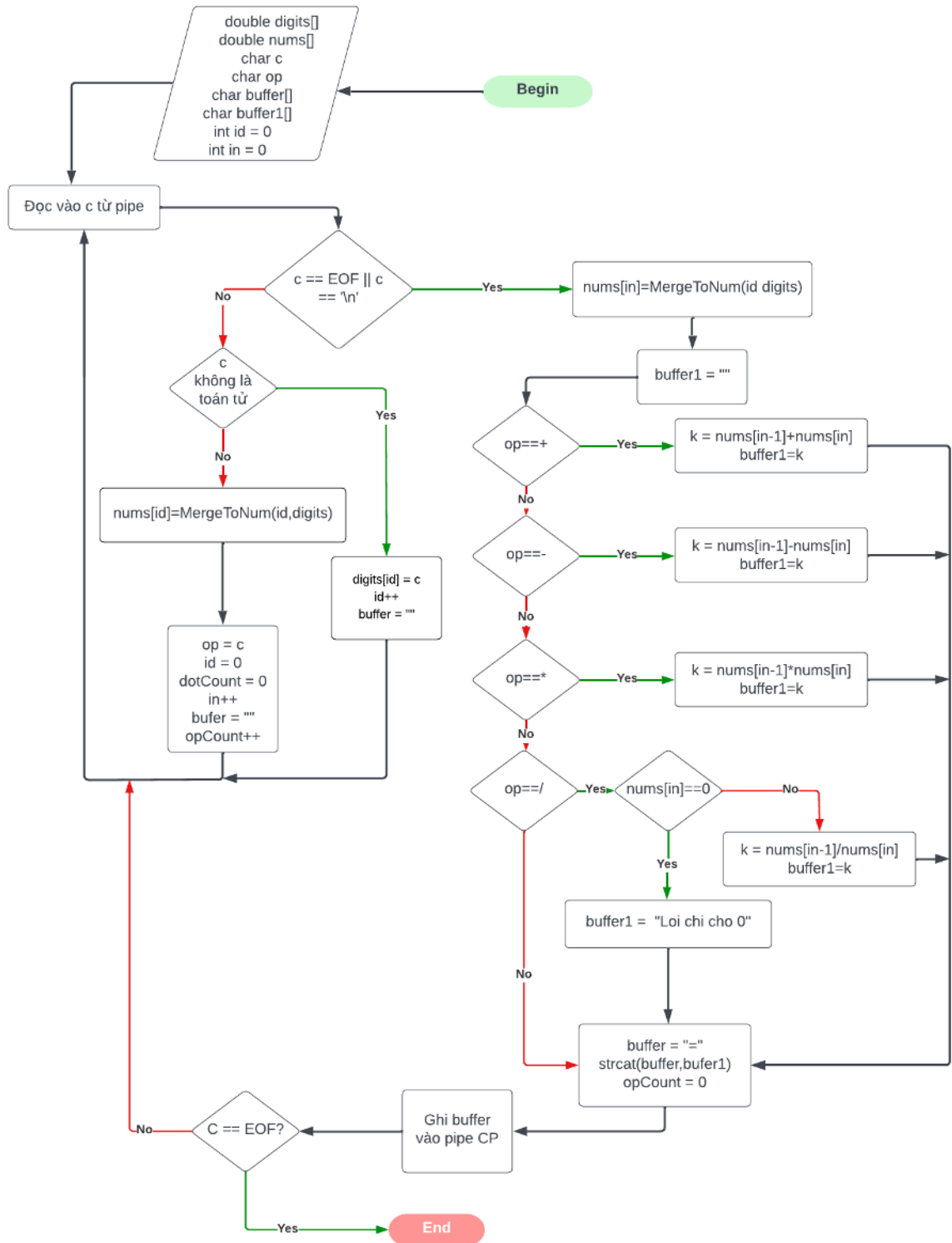


Hình 8. Hàm gộp các thành phần của mảng số thực thành một số thực.

Như vậy hàm `MergeToNum()` có đối số là một mảng số thực với một chỉ số `_index`. Hàm này có chức năng gộp các chữ số trong mảng từ chỉ số 0 đến chỉ số

_index -1 lại thành một số thực.

Sử dụng hàm MergeToNum(), ta có được thuật toán xử lý dữ liệu của tiến trình con như sau:



Hình 9. Thuật toán xử lý dữ liệu của tiến trình con

c. Thuật toán xử lý các biệt lệ:

Dựa vào cấu trúc tập tin nguồn vào chúng ta có một số các biệt lệ như sau:

- Các ký tự đọc không phải chữ số, toán tử hay dấu chấm.
- Các toán tử đọc vào hay ký tự dấu chấm chia phần thập phân bị dư hoặc không theo đúng quy tắc.
- Nhập thiếu một trong hai số trước hoặc sau toán tử.
- Trên một số nền tảng, ví dụ Window Subsystem for Linux thì nếu file có ký tự xuống dòng, Window sẽ đọc thành hai ký tự là `\r` và `\n`. Nhưng ở trên Linux chỉ là `\n`.

Dựa vào đó, chúng ta có thuật toán xử lý các biệt lệ như sau:

- Ở tiến trình cha, ta chỉ xử lý biệt lệ về ký tự `\r`. Nếu đọc phải ký tự này ta sẽ bỏ qua để đọc tiếp mà không ghi nó vào trong pipe.
- Ở tiến trình con, nếu ký tự đọc vào không phải là toán tử, và không phải là số thì có hai trường hợp
 - i. Ký tự đọc vào là dấu chấm (.). Khi đó ta xét xem đây có là dấu chấm đầu tiên được đọc cho một số hay không. Nếu có thì ghi lại để xử lý tính toán, nếu không là lỗi
 - ii. Ký tự đọc vào không là dấu chấm. Đây là lỗi.
- Nếu tiến trình con đọc vào một toán tử thì có rất nhiều trường hợp biệt lệ cần phải xét.
 - i. Nếu toán tử là (+) hoặc (-).
 - Nếu nó đứng đầu một hàng mới, không xảy ra lỗi, không lưu lại toán tử
 - Nếu trước đó đã lưu toán tử của phép tính và toán tử đọc được đứng liền ngay sau ký tự toán tử đó, đây cũng không phải lỗi

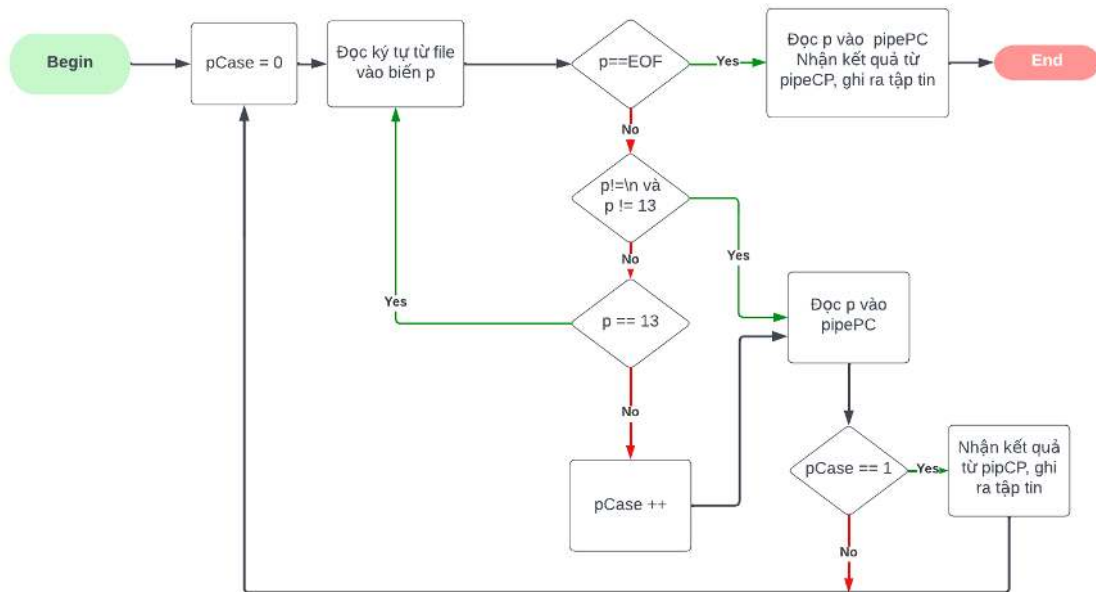
- Nếu trước đó đã lưu toán tử của phép tính và toán tử đọc được không đứng liền sau ký tự toán tử đó, đây là lỗi nhập liệu
- Nếu trước chưa lưu toán tử dùng để thực hiện phép tính và toán tử hiện tại không đứng đầu hàng mới thì đây chính là toán tử để tính toán.
- ii. Nếu toán tử là (*) hoặc (/).
 - Nếu toán tử này đứng đầu dòng mới, xảy ra lỗi
 - Nếu đã có toán tử phép tính, xảy ra lỗi

- Nếu tiến trình con phát hiện nhập thiếu một trong hai số, xảy ra lỗi.

Ta có sơ đồ thuật toán xử lý biệt lệ cho từng tiến trình như sau:

Đối với tiến trình cha, ta thấy có bốn trường hợp xảy ra:

- Trường hợp chỉ đọc ghi ký tự từ tập tin đích mà không đọc ký tự từ pipe
- Trường hợp đọc ghi ký tự từ tập tin đích và nhận kết quả từ pipe sau đó tiếp tục vòng lặp
- Trường hợp đọc ghi ký tự từ tập tin đích và nhận kết quả từ pipe sau đó kết thúc tiến trình
- Trường hợp không làm gì cả, đọc tiếp ký tự tiếp theo



Hình 10. Thuật toán xử lý biệt lệ tiến trình cha

Đối với tiến trình con, ứng với ba trường hợp xử lý dữ liệu ta cũng có ba trường hợp xảy ra biệt lệ nhưng có rất nhiều cách để dẫn đến chúng.

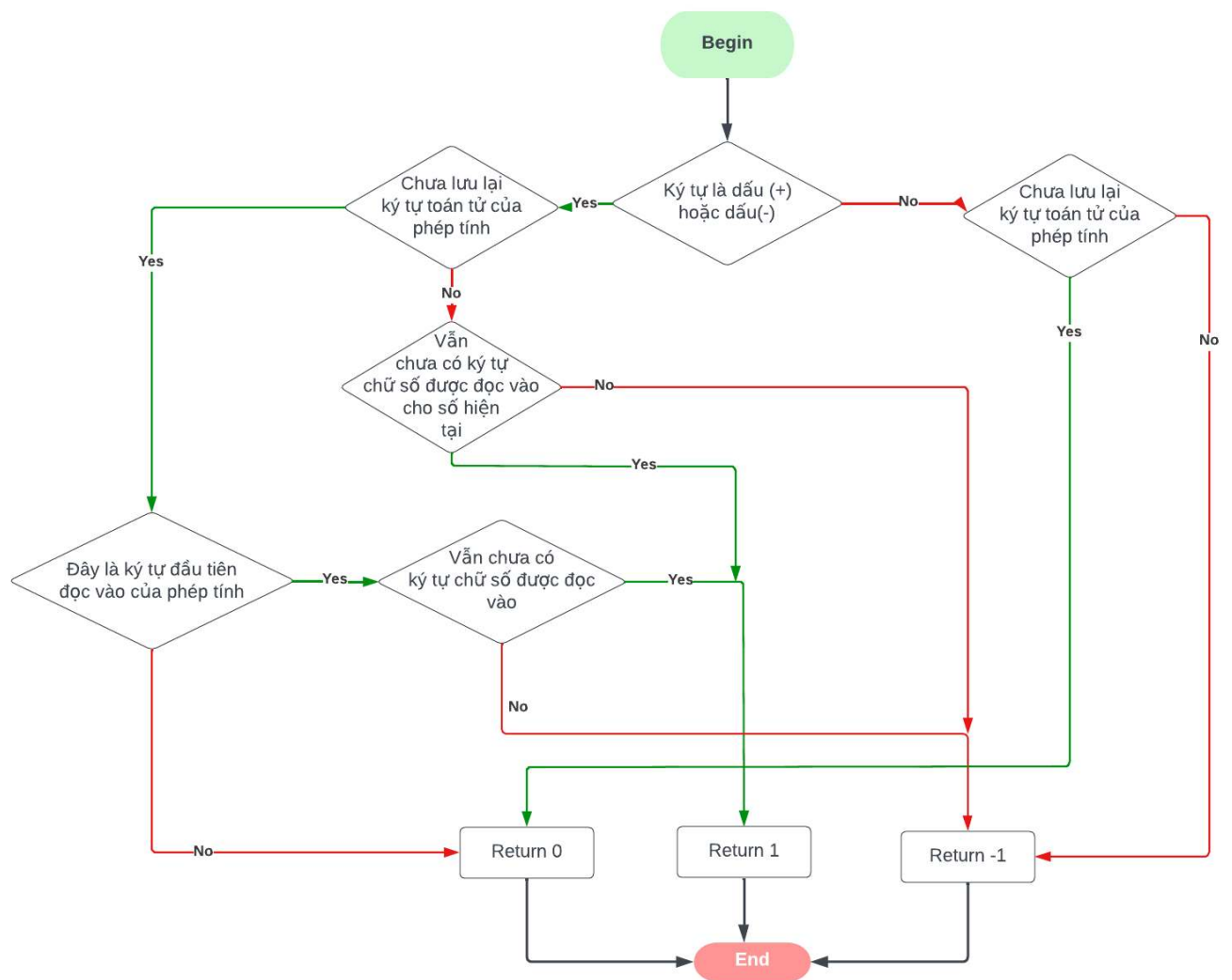
- Trường hợp 1 không lỗi, nhưng chỉ đọc và lưu ký tự hoặc lưu số.
- Trường hợp 2 không lỗi, nhưng phải tổng hợp lại các số đã lưu.
- Trường hợp 3 xảy ra lỗi nhập liệu.

Ta sẽ sử dụng một biến là *case* để xác định được các trường hợp này!

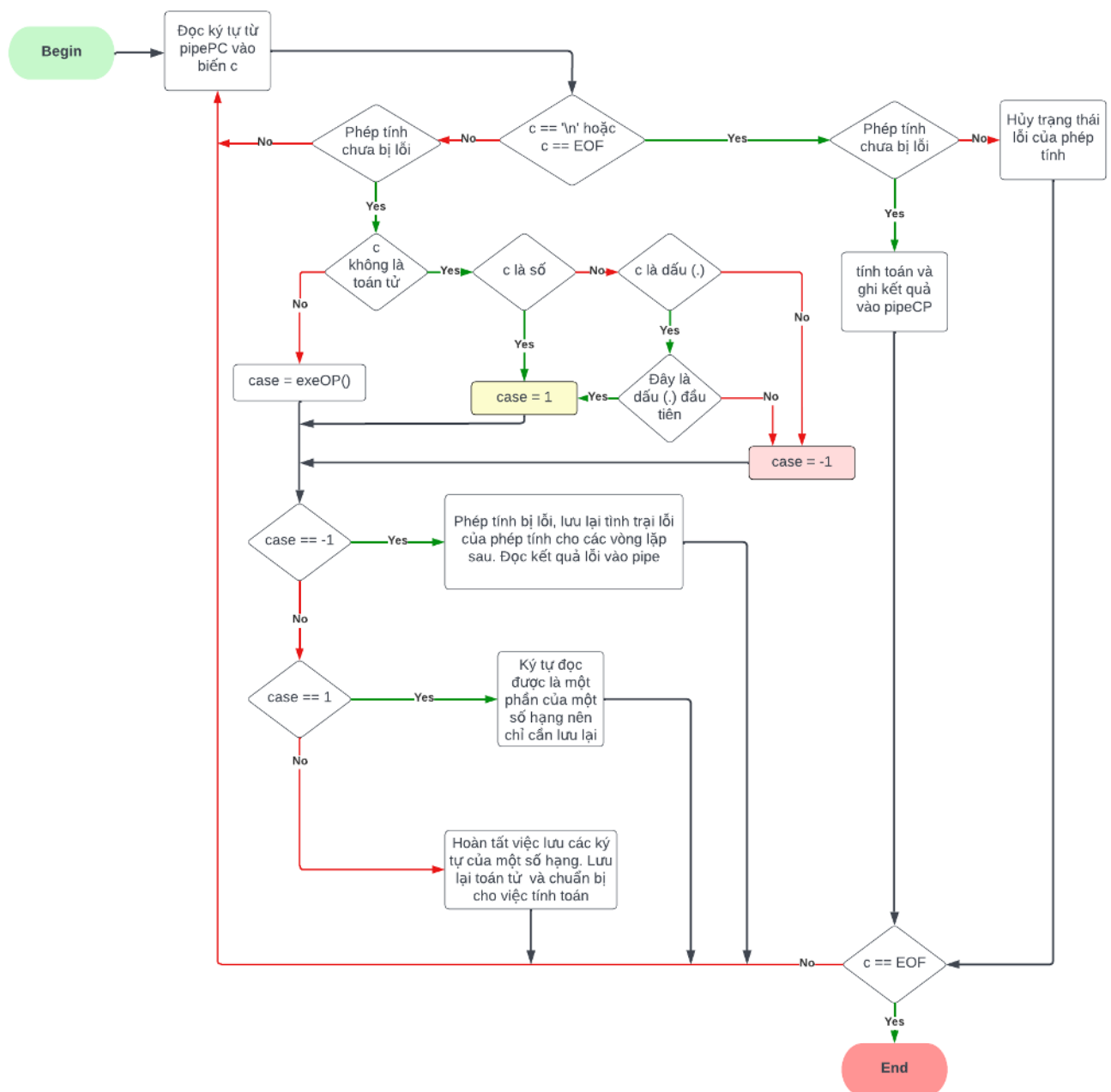
- Case = 1 : trường hợp 1
- Case = 0: trường hợp 2
- Case = -1: trường hợp 3 cũng chính là lỗi.

Nếu ký tự nhận được là toán tử, ta có riêng một hàm `exeOP()` trả về một trong ba số nguyên trong tập $\{0, 1, -1\}$ để xử lý biệt lệ như hình 11.

Tổng hợp lại ta có thuật toán xử lý biệt lệ của tiến trình con như hình 12:



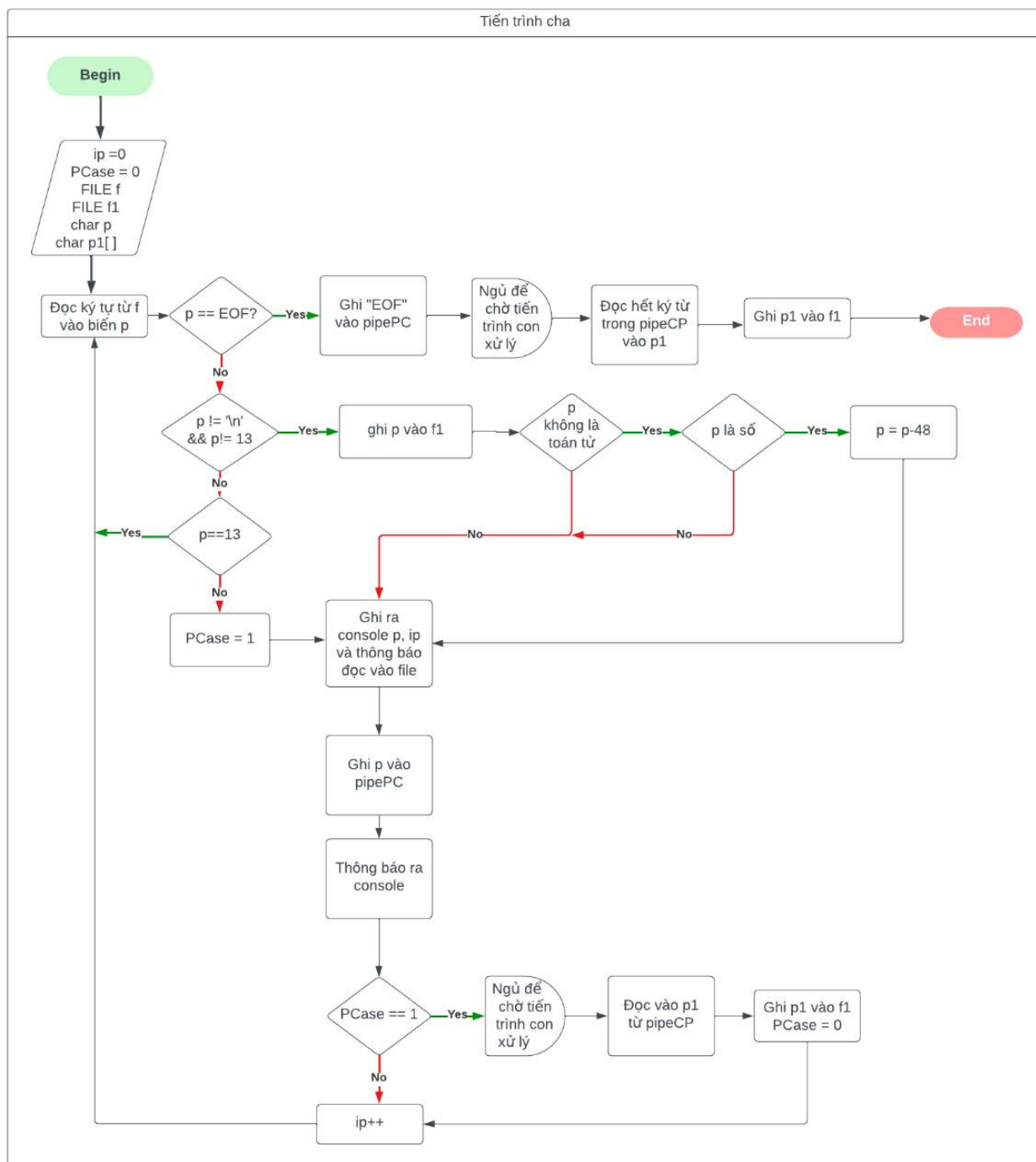
Hình 11. Hàm `exeOP()` xử lý biệt lệ khi ký tự nhận được là một toán tử



Hình 12. Thuật toán xử lý biệt lệ tiến trình con

d. Thuật toán chi tiết tiến trình cha

Tổng hợp các thuật toán đồng bộ, xử lý dữ liệu và xử lý biệt lệ, ta có thuật toán chi tiết của tiến trình cha:



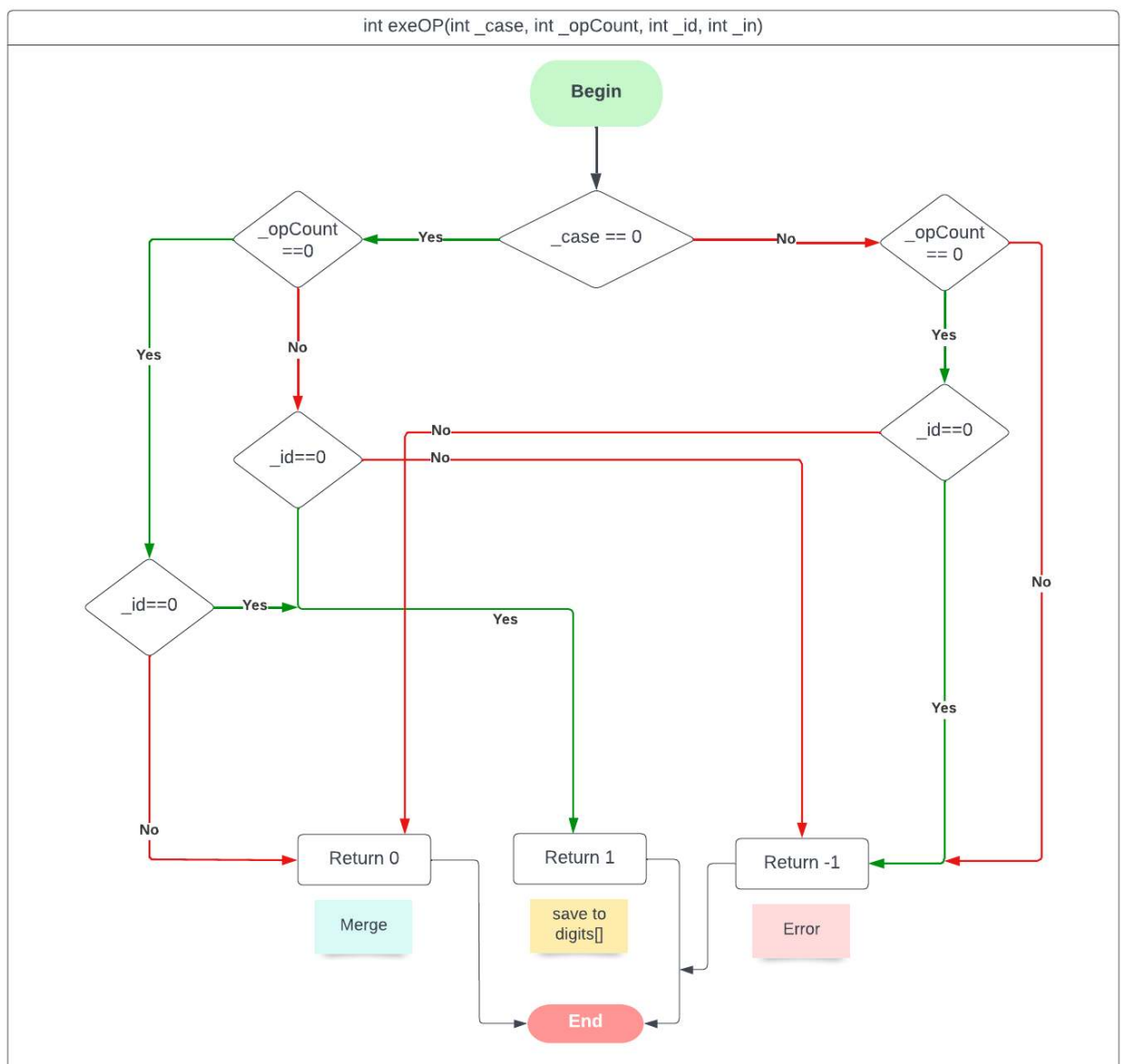
Hình 13. Thuật toán tiến trình cha

e. Thuật toán chi tiết tiến trình con

Tổng hợp lại thuật toán đồng bộ, thuật toán xử lý dữ liệu và thuật toán xử lý biệt lệ ta có được thuật toán chi tiết của tiến trình con bao gồm hai hàm exeOP(), MergeToNum() và thuật toán chính.

Thuật toán hàm MergeToNum() như ở hình 8, trang 11

Thuật toán hàm exeOP():



Hình 14. Hàm `exeOP()` xử lý biệt lệ cho ký tự toán tử

Thuật toán chi tiết tiến trình con:

IV. Demo chương trình:

a. Mã nguồn tổng quan chương trình:

```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <time.h>

#include <string.h>

#include <math.h>

#define MAXSIZE 100

//Hàm xử lý biệt lệ khi ký tự là một toán tử

int exeChar(int _case, int _opCount, int _id, int _in)

{

    printf("\t\tcase = %d ", _case);

    printf("\t\ttopCount=%d; ", _opCount);

    printf("\t\tid = %d ", _id);

    printf("\t\tin = %d \n", _in);

    if (_case == 0) // dấu + hoặc -

    {

        //Nếu chưa có phép toán

        if (_opCount == 0)

        {
```

```

        //Xét xem nó phải là dấu của số

        //Nếu có thì thêm vào số

        if (_id == 0)
        {
            return 1;
        }

        //Nếu không thì đó là phép tính

        else if (_id != 0)
        {
            return 0;
        }
    }

    //Nếu đã tồn tại phép toán

    else
    {
        //Nếu đó là dấu của số

        if (_id == 0)
        {
            return 1;
        }

        //Nếu không là lỗi

        else
        {
            return -1;
        }
    }
}

else // Dấu * hoặc /
{

```



```

        if (_opCount == 0)
        {
            if (_id == 0)
                return -1;

            else
                return 0;
        }

        else
            return -1;
    }
}

// -----

// Hàm gộp các chữ số trong một mảng thành một số thực
double Merge2Num(int index, double *a)
{
    double d = 0;

    int t = 0;

    int isNegative = 0;

    double hasDecimal = 0;

    // Kiểm tra xem phần tử đầu tiên trong mảng có phải là dấu
    không

    switch ((int)*a)

```

```

{
    case 43: // Trường hợp dấu +
        t++;
        break;
    case 45: // Trường hợp dấu -
        isNegative = 1;
        t++;
        break;
    default:
        break;
}

```

```

// tiến hành gộp
while (t < index)
{
    // trường hợp phần nguyên
    if (hasDecimal == 0)
    {
        if (*(a + t) == 46)
            hasDecimal++;
        else
        {
            d *= 10;
            d += *(a + t);
        }
    }
}

```

```

    }

    // trường hợp phần thập phân
    else
    {
        int pow = 1;

        for (int j = 0; j < hasDecimal; j++)
        {
            pow *= 10;
        }

        d += *(a + t) / pow;

        hasDecimal++;
    }

    t++;
}

// Kiểm tra là số âm hay số dương để return
if (isNegative == 1)
    return -d;

return d;
}

// -----

// Hàm chính
int main(int argc, char *argv[])
{

```

```

int pipeCP[2]; // pipe giao tiep từ con sang cha
int pipePC[2]; // pipe giao tiếp từ cha sang con


// tạo pipe từ các mảng pipe
if (pipe(pipeCP) == -1)
{
    printf("pipeCP error!");
    return 1;
}
if (pipe(pipePC) == -1)
{
    printf("pipePC error!");
    return 1;
}

// tạo tiến trình với hàm fork
int pid = fork();
if (pid == -1)
{
    printf("pid Error!");
    return 2;
}

// bắt đầu lập trình cho từng tiến trình
if (pid != 0) // tiến trình cha

```

```

{
    // đóng đầu ghi của pipeCP và đầu đọc của pipePC
    close(pipeCP[1]);
    close(pipePC[0]);

    //----Mã nguồn tiến trình cha----

    // Đóng đầu đọc của pipeCP và đầu ghi của pipePC
    close(pipeCP[0]);
    close(pipePC[1]);
}
else // tiến trình con
{
    // Đóng đầu đọc của pipeCP và đầu ghi của pipePC
    close(pipeCP[0]);
    close(pipePC[1]);

    //-----Mã nguồn tiến trình con----

    // Đóng đầu ghi của pipeCP và đầu đọc của pipePC
    close(pipeCP[1]);
    close(pipePC[0]);
}
}

```

b. Mã nguồn tiến trình cha:

```

char p;      // biến lưu trữ ký tự ghi vào pipePC

char p1[MAXSIZE];    // biến nhận chuỗi thông tin từ pipeCP

FILE *f = fopen("readFile.txt", "r"); // file để đọc các phép tính

FILE *f1 = fopen("writtenFile.txt", "a"); // file để ghi kết quả

int ip = 0; // biến lưu trữ số ký tự đã đọc

int pCase = 0;    // biến trạng thái xác định trường hợp biệt

lệ

while (1)

{

    p = (char)fgetc(f); // đọc ký tự từ file f

    if (p == EOF) // Nếu đó là ký tự kết thúc file (End Of
File):

    {

        printf("lần đọc thứ %d p = EOF \n", ip);

        // ghi EOF vào pipePC để báo cho tiến trình con

        if (write(pipePC[1], &p, sizeof(p)) == -1)

            return -1;

        sleep(0.5);    // ngủ để chờ tiến trình con xử lý

        // nhận giá trị từ tiến trình con ở pipeCP:

        if (read(pipeCP[0], &p1, sizeof(p1)) == -1)

            return 2;

        printf("Cha nhan %s tu pipeCP\n", p1);

        fputs(p1, f1); // đọc giá trị nhận được vào file f1

        fflush(f1);    // xóa bộ nhớ tạm của f1

```

```

        printf("Cha ghi %s vao file\n", p1);

        break; // Kết thúc tiến trình cha
    }

    else // Nếu p không là ký tự kết thúc file:
    {
        // Nếu p không là ký tự xuống dòng hoặc ký tự hồi trả con trỏ

        if (p != '\n' && p != 13)
        {
            fputc(p, f1); // ghi ký tự p vào file f1

            // Chuyển ký tự số thành số.

            if (p >= '0' && p <= '9')
                p = p - 48;
        }

        else // Nếu p là ký tự xuống dòng hoặc hồi trả con trỏ
        {
            // Nếu p là hồi trả con trỏ thì thi bỏ qua

            if (p == 13)
                continue;

            else // Nếu p là ký tự xuống dòng thì đánh dấu pCase

                pCase = 1;
        }

        // Tiến hành ghi p vào pipePC:

        printf("lần đọc thứ %d", ip);
    }

```

```

printf(" p = %d\n", p);

printf("Cha ghi %d vao file\n", p);

if (write(pipePC[1], &p, sizeof(p)) == -1)

    return 1;

printf("Cha ghi %d vao pipePC\n", p);


// Nếu pCase được đánh dấu

if (pCase == 1)

{

    sleep(1); // Ngủ 1s để chờ tiến trình con xử lý

    // Đọc thông tin từ tiến trình con:

    if (read(pipeCP[0], &p1, sizeof(p1)) == -1)

        return 2;

    fputs(p1, f1); // Ghi thông tin nhận được vào file

f1

    fflush(f1);    // xóa bộ nhớ đệm

    printf("Cha ghi %s vao file\n", p1);

    pCase = 0; // Xóa đánh dấu cho pCase

}

ip++;

}

}

```

c. Mã nguồn tiến trình con.

```
double digits[MAXSIZE]; // Mảng lưu các chữ số của một số
```



```

double nums[MAXSIZE]; // Mảng lưu các số của phép tính

char c;                // Biến đọc giá trị từ pipePC

char op;               // Biến lưu toán tử

char buffer[MAXSIZE]; // Biến thông tin để ghi vào pipeCP

char buffer1[MAXSIZE]; // Biến hỗ trợ lưu thông tin

int id = 0;            // Biến index cho digits

int in = 0;            // Biến index cho nums

int calError = 0;      // Biến xác định trạng thái lỗi của phép tính

int opCount = 0;       // Biến xác định đã đọc ký tự phép toán

int theCase = 0;       // Biến xác định trường hợp xử lý ký tự

int dotCount = 0;      // Biến đếm số dấu (.)

while (1)
{
    // đọc ký tự từ pipePC

    if (read(pipePC[0], &c, sizeof(c)) == -1)
    {
        printf("\tREAD PIPEPC ERROR!");
        return 1;
    }

    printf("\tc= %d \n", c);

    /*Nếu c không phải ký tự kết thúc file cũng không phải \n
=> thu thập thông tin ký tự */

    if (c != EOF && c != '\n')
    {
        // Nếu phép tính đó không bị lỗi

```

```

if (calError == 0)
{
    printf("\tcalError == 0\n");

    // Nếu c không là toán tử
    if (c != '+' && c != '-' && c != '*' && c != '/')
    {
        // Nếu c là số => rơi vào case = 1
        if ((c >= 0 && c <= 9))
            theCase = 1;
        else if (c == '.') // Nếu c là dấu (.)
        {
            // Nếu chưa tồn tại dấu .
            //=> rơi vào case = 1
            if (dotCount == 0)
            {
                dotCount++;
                theCase = 1;
            }
            // Nếu đã tồn tại => lỗi
            //=> rơi vào case = -1
            else
                theCase = -1;
        }
        else // c là ký tự khác => lỗi

```

```

        theCase = -1;
    }

    else // c là toán tử, chạy hàm exeChar()
    {
        if (c == '+' || c == '-')
        {

            theCase = exeChar(0, opCount, id, in);

            printf("\tThe case = %d\n", theCase);
        }

        else
        {
            theCase = exeChar(1, opCount, id, in);

            printf("\tThe case = %d\n", theCase);
        }
    }
}

// Xét các trường hợp xử lý ký tự
switch (theCase)
{
    case -1: // trường hợp lỗi
    {
        // Thông báo lỗi, đánh dấu phép tính bị lỗi
        calError = 1;

        strcpy(buffer, ": ");
    }
}

```

```

        strcat(buffer, "Loi nhap lieu\n");

        // Ghi lỗi vào pipeCP

        if (write(pipeCP[1], &buffer,
        sizeof(buffer)) == -1)

        {

            printf("\tWRITING PIPECP ERROR!");

            return 1;

        }

        printf("\tCon ghi %s vao pipeCP\n", buffer);

        id = 0;

        dotCount = 0;

        in = 0;

        break;

    }

    case 1: // trường hợp case = 1;

    {

        // Chỉ đọc thêm ký tự vào chuỗi digits

        digits[id] = c;

        printf("\tdigits[%d]=%f\n", id, digits[id]);

        strcpy(buffer, "");

        id++;

        break;

    }

    case 0: // trường hợp case = 0

```

```

{
    // gộp các ký tự trong mảng thành một số
    nums[in] = Merge2Num(id, digits);

    printf("\tnums[%d] = %f\n", in, nums[in]);

    // lưu lại toán tử của phép tính

    op = c;

    opCount++;

    strcpy(buffer, "");

    id = 0;

    dotCount = 0; // đặt lại trạng thái dotcount

    in++;

}

default:

    break;

}

}

}

else // Nếu c là EOF hoặc xuống dòng

{

    // Nếu phép tính không bị lỗi

    if (calError == 0)

    {

        printf("\tcalError == 0\n");

        // Tiến hành tính toán rồi phản hồi lại

```

```
// tiến hành gộp các chữ số trong mảng lại  
thành một số
```

```
nums[in] = Merge2Num(id, digits);  
  
printf("\tnums[%d] = %f\n", in, nums[in]);  
  
// Tính toán theo toán tử đã lưu lại  
  
strcpy(buffer1, "");  
  
switch (op)  
{  
  
case '+':  
  
{  
  
    double k = nums[in] + nums[in - 1];  
  
    printf("\tk = %f", k);  
  
    // chuyển đổi số thực k thành một string  
    có tối đa 5 chữ số  
  
    gcvt(k, 5, buffer1);  
  
    break;  
  
}  
  
case '-':  
  
{  
  
    double k = nums[in - 1] - nums[in];  
  
    printf("\tk = %f\n", k);  
  
    gcvt(k, 5, buffer1);  
  
    break;  
  
}
```

```

case '*':
{
    double k = nums[in] * nums[in - 1];

    printf("\tk = %f\n", k);

    gcvt(k, 5, buffer1);

    break;
}

case '/':
{
    if (nums[in] == 0)
    {
        strcat(buffer1, "Loi chia cho 0");
    }

    else
    {
        double k = nums[in - 1] / nums[in];

        printf("\tk = %f\n", k);

        gcvt(k, 5, buffer1);
    }

    break;
}

default:
    break;
}

```

```

// Ghi kết quả vào pipeCP

strcpy(buffer, "=");

strcat(buffer, buffer1);

strcat(buffer, "\n");

if (write(pipeCP[1], &buffer, sizeof(buffer)) == -1)
{
    printf("\tWRITING PIPECP ERROR!");

    return 1;
}

printf("\tCon ghi %s vào pipeCP\n", buffer);

id = 0;

dotCount = 0;

in = 0;

opCount = 0;
}

else // Nếu phép tính bị lỗi, reset lại trạng thái lỗi
{
    calError = 0;

    opCount = 0;
}

/*Nếu c là EOF

=> Tiến trình cha đã đọc hết file

=> Kết thúc tiến trình con*/

if (c == EOF)
{

```



```

        printf("\tCon nhan EOF tu pipePC\n");

        break;
    }

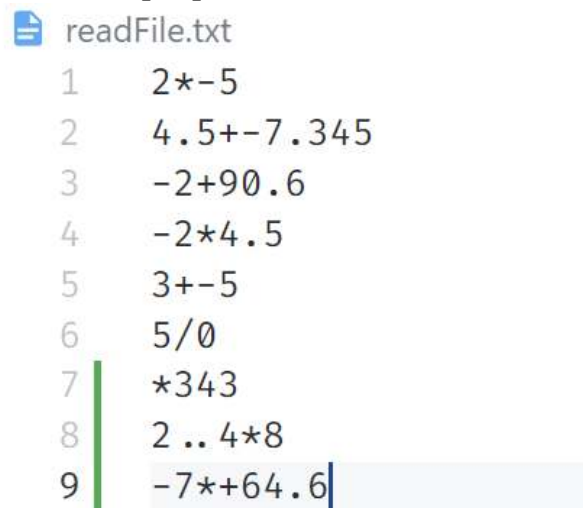
}

}

```

d. Kết quả chạy chương trình.

Ta có file *readFile.txt* lưu các phép tính sau:



```

readFile.txt
1      2*-5
2      4.5+-7.345
3      -2+90.6
4      -2*4.5
5      3+-5
6      5/0
7      *343
8      2 .. 4*8
9      -7*+64.6

```

Hình 16. Thông tin phép tính

Sau khi chạy chương trình ta có được kết quả từ file *writtenFile.txt* như sau

```
writtenFile.txt
1 2*-5=-10
2 4.5+-7.345=-2.845
3 -2+90.6=88.6
4 -2*4.5=-9
5 3+-5=-2
6 5/0=Loi chia cho 0
7 *343: Loi nhap lieu
8 2..4*8: Loi nhap lieu
9 -7*+64.6=-452.2
10
```

Hình 17. Kết quả tính toán

```

store.c 2 M  writtenFile.txt M  readFile.txt M  PipeCal - Visual Studio Code
writtenFile.txt
1 2*-5=-10
2 4.5+-7.345=-2.845
3 -2+90.6=88.6
4 -2*4.5=-9
5 3+-5=-2
6 5/0=Loi chia cho 0
7 *343: Loi nhap lieu
8 2..4*8: Loi nhap lieu
9 -7*+64.6=-452.2
10

readFile.txt
1 2*-5
2 4.5+-7.345
3 -2+90.6
4 -2*4.5
5 3+-5
6 5/0
7 *343
8 2..4*8
9 -7*+64.6

PipeCal
PS T:\Code\C\PipeCal> bash
letritan@vialius:~/mnt/t/Code/C/PipeCal$ ./store
lần đọc thứ 0 p = 2
Chia ghi 2 vào file
Chia ghi 2 vào pipePC
lần đọc thứ 1 p = 42
Chia ghi 42 vào file
Chia ghi 42 vào pipePC
lần đọc thứ 2 p = 45
Chia ghi 45 vào file
Chia ghi 45 vào pipePC
lần đọc thứ 3 p = 5
Chia ghi 5 vào file
Chia ghi 5 vào pipePC
lần đọc thứ 4 p = 10
Chia ghi 10 vào file
Chia ghi 10 vào pipePC
c = 2
calError == 0
digits[0]=2.000000
c = 42
calError == 0
case = 1          opCount=0;          id = 1 i
n = 0
The case = 0
nums[0] = 2.000000
c = 45
calError == 0
case = 0          opCount=1;          id = 0 i
n = 1
The case = 1
digits[0]=45.000000
c = 5
calError == 0
digits[1]=5.000000
c = 10
calError == 0
nums[1] = -5.000000
k = -10.000000
Con ghi --10
vao pipeCP
Chia ghi --10
vao file
lần đọc thứ 5 p = 4
Chia ghi 4 vào file

```

Hình 18. Kết quả chạy và thông tin xuất ra console

PHẦN 2: QUẢN TRỊ MẠNG

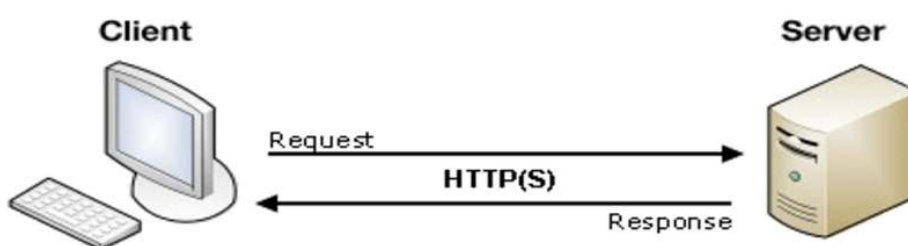
I. Cơ sở lý thuyết:

a. Mô hình Client-Server:

Mô hình Client-Server là một trong số các mô hình tổ chức mạng máy tính, bên cạnh các mô hình khác là mô hình mạng ngang hàng (Peer-to-Peer) hay mô hình mạng lai (Hybrid). Thuật ngữ Client/Server xuất hiện lần đầu vào những năm 80 của thế kỉ trước.

Mô hình Client-Server là mô hình mạng cơ bản và phổ biến nhất hiện nay. Nó là dạng phổ biến nhất của mô hình ứng dụng phân tán. Đa số ứng dụng mạng được xây dựng dựa trên mô hình này, tiêu biểu có thể kể tên như ứng dụng Email, ứng dụng web, ứng dụng truyền file theo giao thức FTP.

Mô hình Client-Server cung cấp một cách tiếp cận tổng quát để chia sẻ tài nguyên trong các hệ thống phân tán. Máy cung cấp tài nguyên sẽ là Server và máy yêu cầu tài nguyên sẽ là Client. Một máy có thể là cả Server lẫn Client khi mà tiến trình server và tiến trình client có thể chạy trên cùng một máy tính và một Server cũng có thể yêu cầu dịch vụ từ một Server khác nữa.

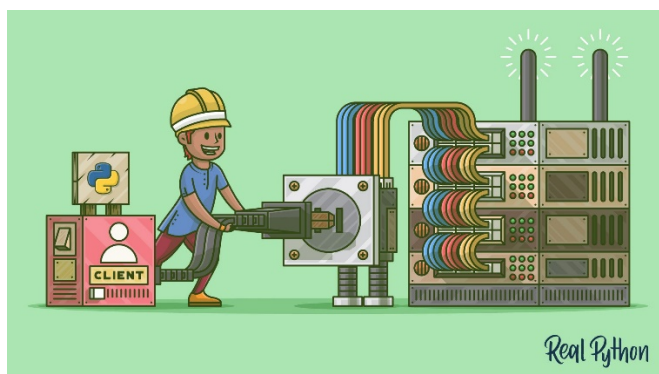


Hình 19. Client gửi yêu cầu và Server phản hồi.

Trong mô hình Client-Server, Client chủ động liên hệ đến Server và đưa ra yêu cầu thiết lập kết nối (request). Server nhận request, chấp nhận yêu cầu kết nối và tạo ra một cầu nối để truyền tải thông tin, đáp ứng nhu cầu của Client. Với cách thiết lập như vậy, ta thấy rằng mô hình truyền tin này là mô hình truyền thông hai chiều, khi mà tiến trình server và tiến trình client phải được đồng bộ hóa với nhau. Tiến trình server phải nhận thức được thông điệp ngay khi yêu cầu được gửi đến, đồng thời tiến trình client cũng phải tạm ngừng phát yêu cầu, chờ cho đến khi nó nhận được phản hồi từ server gửi về.

b. Mô hình truyền tin Socket:

Socket là một giao diện lập trình ứng dụng mạng được dùng để truyền và nhận dữ liệu trên internet. Giữa các thiết bị kết nối với nhau sẽ có một kênh truyền thông hai chiều và hai điểm cuối của kênh truyền thông này ở hai máy chính là socket. Có thể hiểu socket là những cái ổ cắm điện có chức năng là đầu cuối giao tiếp trong một kết nối.



Hình 20. Socket là những cái ổ cắm điện

Trong mô hình Client-Server thì socket giúp kết nối giữa Client và Server thông qua giao thức TCP/IP hoặc UDP. Socket chỉ có thể hoạt động khi nắm được thông tin địa chỉ IP của các thiết bị và số hiệu cổng làm việc của ứng dụng cần nhận được gói tin chạy trên các thiết bị đó.

Một Socket có thể thực hiện được bảy thao tác cơ bản:

1. *Kết nối với một máy ở xa*
2. *Gửi dữ liệu đi*
3. *Nhận dữ liệu về*
4. *Ngắt kết nối*
5. *Gán cổng*
6. *Nghe dữ liệu đến*
7. *Chấp nhận liên kết từ các máy ở xa trên cổng được gán*

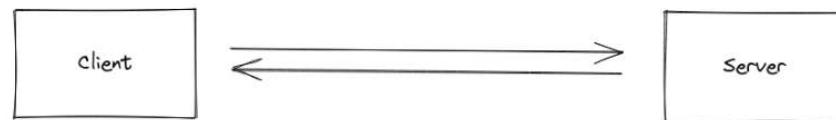
Socket được sử dụng trên hầu hết các ngôn ngữ lập trình và tương thích với mọi cấu hình máy tính khác nhau.

c. Socket.IO:

Để có thể tạo một mô hình Client-Server trên nền tảng web sử dụng ngôn ngữ lập trình JS, ta có thể sử dụng một thư viện có tên Socket.IO.

Socket.IO là một thư viện được phát triển dựa trên nền tảng của một công nghệ khác là Websocket. Websocket là một giao thức truyền tải ra đời để thay thế cho giao thức HTTP truyền thống. Websocket cung cấp liên kết hai chiều rất mạnh mẽ, có độ trễ thấp hơn so với HTTP và dễ sửa lỗi. Websocket thường được ứng dụng cho các trường hợp cần độ trễ thấp như ứng dụng chat, ứng dụng mua bán, giao dịch tiền tệ. Dù có độ trễ thấp nhưng request từ Websocket không tốn nhiều tài nguyên, traffic như HTTP.

Socket.IO thừa kế những ưu điểm của Websocket với độ trễ cao, giao tiếp hai chiều và giao tiếp dựa trên sự kiện. Nghĩa là trong quá trình Client và Server kết nối với nhau. Client có thể xuất sự kiện cho Server lắng nghe xử lý và Server cũng có thể xuất một sự kiện để Client lắng nghe và xử lý!



Hình 21. Giao tiếp hai chiều giữa Client và Server

Socket.IO hỗ trợ rất nhiều ngôn ngữ lập trình khác nhau như Java, C++, Python hay Java Script. Trong đề tài lần này ta sử dụng ngôn ngữ Java Script.

Đầu tiên ta cần cài đặt thư viện Socket.io trên cả máy Server lẫn Client.

Bên phía Server: `npm install socket.io`

Bên phía Client, ta có thể dùng:

```
<script src="/socket.io/socket.io.js"></script>
```

```
<script>
```

```
const socket = io();
```

```
</script>
```

Hoặc có thể cài đặt từ NPM: `npm install socket.io-client`

Để bắt đầu kết nối, phía Server ta sẽ tạo ra một luồng vào ra với cổng 3000 làm ví dụ như sau:

```
import { Server } from "socket.io";
```

```
const io = new Server(3000);
```

Sử dụng phương thức `.on()` để xác nhận khi Server được kết nối, đối số là “connection” và một callback nhận về dữ liệu là socket từ phía client. Mọi hoạt động giao tiếp với client sẽ được lập trình trong callback này,

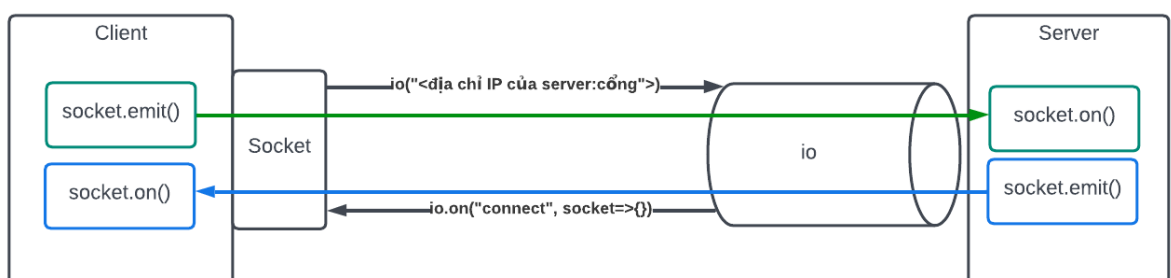
```
io.on("connection", (socket) => {  
  
    // code here  
  
});
```

Về phía Client, ta sẽ tạo một socket để giao tiếp với Server. Chuỗi nhập vào sẽ là địa chỉ IP của Server cần kết nối, trong ví dụ này ta đang sử dụng mạng localhost, cổng 3000 cho Server

```
import { io } from "socket.io-client";  
  
const socket = io("ws://localhost:3000");
```

Trong cách thức giao tiếp này, Client và Server giao tiếp với nhau thông qua việc lắng nghe các sự kiện. Giống như phương thức `addEventListener()` của JavaScript, Socket.IO cung cấp hai phương thức là `socket.emit()` và `socket.on()`. Phương thức `.emit()` sẽ lắng nghe các sự kiện và trả về cho bên kia một dữ liệu nào đó. Bên còn lại sử dụng phương thức `.on()` để bắt được đúng sự kiện, nhận giá trị cung cấp và thực hiện yêu cầu.

Tổng quan lại, ta có sơ đồ giao tiếp bằng Socket.IO một cách tổng quan nhất như sau:



Hình 22. Tổng quan việc kết nối với Socket.IO

d. Sơ lược về RSS:

RSS là một định dạng tập tin thuộc họ XML được sử dụng cho việc chia sẻ nội dung website. Có rất nhiều đáp án cho việc RSS là viết tắt của từ nào, nó tùy vào các phiên bản RSS:

- Rich Site Summary (phiên bản RSS 0.91)
- RDF Site Summary (phiên bản RSS 1.0)
- Really Simple Syndication (phiên bản RSS 2.0)

RSS cho phép người dùng theo dõi nội dung từ một trang web có khả năng cung cấp dịch vụ RSS (hay RSS feeds). Các trang web này thường có nội dung được thay đổi và thêm vào thường xuyên. RSS tóm tắt các đặc tính và thông tin về trang web thành một định dạng kiểu XML. Nội dung của các RSS thay đổi theo chính sự thay đổi của nội dung trang web. Vì vậy người sử dụng chỉ cần một lần thêm RSS feed là có thể theo dõi được trang web mà không cần cập nhật lại.

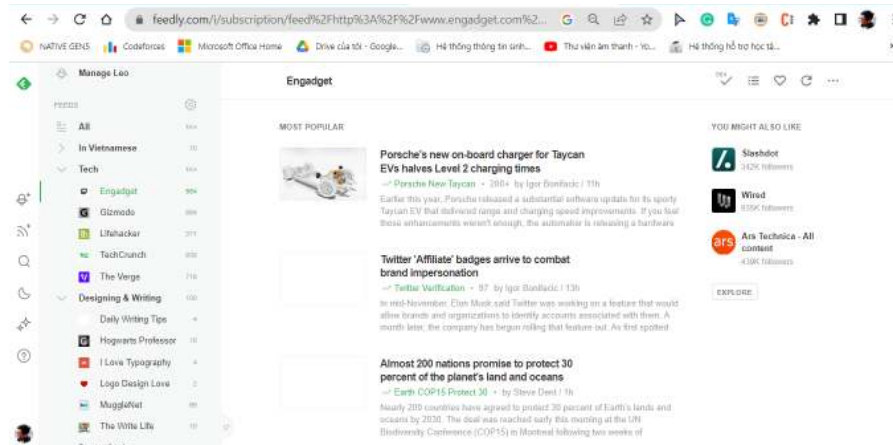
RSS định dạng lại nội dung của trang web theo dạng một danh sách tóm lược các đầu nội dung của trang web đó kèm theo link đến phần nội dung đầy đủ cùng những dữ liệu meta khác. Ví dụ với RSS của một blogger, ta có thể lấy được thông tin của Blogger đó như tên, số lượng bài viết kèm theo một danh sách các bài viết gần đây với phần tóm tắt nội dung và những dữ liệu như ngày đăng, hình minh họa v.v.. Chính vì điều này mà những nhà lập trình có thể xây dựng những ứng dụng đọc RSS để tổ chức và quản lý các kênh theo dõi đó. Một số ứng dụng đọc RSS nổi tiếng như: Feedly, Feedbin, The Old Reader.



Hình 23. Logo của RSS

Những lợi ích của việc sử dụng RSS:

- Đối với người chia sẻ nội dung, RSS giúp chia sẻ thông tin rộng rãi, nhanh chóng mà không tốn chi phí để quảng bá.
- Đối với người tiếp nhận nội dung, RSS giúp theo dõi và cập nhật các nội dung mới một cách nhanh chóng. Các ứng dụng đọc RSS còn giúp phân phối và tổ chức các nguồn thông tin, giúp việc học hỏi trở nên nhanh chóng, dễ dàng nhưng có chọn lọc hơn



Hình 24. Giao diện của Feedly, một ứng dụng đọc tin RSS

II. Mô tả đề tài:

Đề tài phần quản trị mạng của báo cáo này là:

Xây dựng ứng dụng đọc RSS tổng hợp nội dung các kênh trên nền tảng chia sẻ video Youtube sử dụng mô hình Client Server

Công nghệ sử dụng cho đề tài này:

- Về phía Client, xây dựng web dựa sử dụng thư viện ReactJS, ngôn ngữ lập trình JavaScript
- Về phía Server, sử dụng nodeJS với framework Express
- Kết nối giữa Client và Server, sử dụng thư viện Socket.IO cho Javascript
- Về cơ sở dữ liệu, sử dụng MongoDB.

Với đề tài này ta sẽ xây dựng ứng dụng có các chức năng như sau:

- Thêm kênh để theo dõi
- Hiển thị nội dung kênh đã theo dõi
- Bỏ theo dõi kênh

Để có thể thêm kênh ta cần API cung cấp tính năng tìm kiếm kênh Youtube và API RSS Feed để lấy thông tin của kênh Youtube đó. Sau khi lấy được mã RSS của kênh Youtube tương ứng, ta phải xây dựng thuật toán chuyển đổi dữ liệu XML thành JSON để có thể sử dụng với Javascript.

III. Thuật toán chương trình:

a. Youtube Search API:

Để có thể tìm kiếm nội dung trên Youtube, ta sử dụng một thư viện cho javascript là *youtube-search-api*. Thư viện này cung cấp một Promise có tên là *GetListByKeyword()*, cụ thể như sau:

```
const youtubearchapi = require("youtube-search-api");

youtubearchapi.GetListByKeyword(

  "<keywords>",

  [playlist boolean],

  [limit number],

  [options JSONArray]

)
```

Đối số của Promise trên bao gồm:

- Keyword: chính là từ khóa mà bạn muốn tìm kiếm, dạng String
- Playlist boolean: chỉ mục để đánh dấu bạn tìm từng đối tượng hay tìm danh sách phát gồm nhiều đối tượng
- Limit number: số kết quả tối đa trả về
- Options: mảng các tùy chỉnh mà bạn yêu cầu kết quả đạt được. Thường dùng để quy định xem sẽ tìm những gì. Ví dụ nếu đối số này bạn nhập là [{type: "video"}] thì nghĩa là bạn đang tìm kiếm video.

Kết quả trả lại của Promise này là một đối tượng có dạng như sau:

```
{

  items:[],

  nextPage:{

    nextPageToken:"xxxxxxx",

    nextPageContext:{}

  }

}
```

Trong đó mảng items trả lại các phần tử mà chúng ta cần tìm, đối tượng nextPage là đối số của một Promise khác giúp chúng ta tải kết quả tìm

kiếm ra nhiều trang.

Trong ứng dụng của báo cáo này, chúng ta sẽ tìm kiếm các kênh dựa trên nội dung người dùng nhập vào, chúng ta sẽ lấy về các thông tin: *id của kênh*, *thumbnail của kênh*, và *tên kênh* để hiển thị cho người dùng. Mã nguồn như sau:

```
<input type="text" onchange="fetchingData()"
id="inputChannel" />

<script>

    function fetchingData(e) = {

        const limit = 7;

        youtubesearchapi.GetListByKeyword(

            e.target.value,

            false,

            limit,

            [{ type: "channel" }]

        )

        .then((res) => {

            const channels = res.items.map((item) => {

                const channel = {

                    channelId: item.id,

                    thumbnail: item.thumbnail.thumbnails[1].url,

                    title: item.title,

                };

                return channel;

            });

        });

    };

</script>
```

b. Youtube RSS Feed Extension

Bản thân Youtube không cung cấp RSS link cho các kênh nội dung trên nền

tảng này. Tuy nhiên có một tiện ích trên chrome cung cấp cho chúng ta một API để lấy được RSS các kênh trên youtube. Link API như sau:

https://www.youtube.com/feeds/videos.xml?channel_id=<>

Chỉ cần thêm id của kênh youtube mà chúng ta đã lấy được vào cuối link API này, chúng ta sẽ nhận được tài liệu RSS có dạng như sau:

```
<feed
  xmlns:yt="..."
  xmlns:media="..."
  xmlns="..."
>

  <link
    rel="self"
    href="link to channel"
  />

  <id>id của kênh</id>

  <yt:channelId>id của kênh</yt:channelId>

  <title>Tên kênh</title>

  <link
    rel="alternate"
    href="link dẫn đến kênh"
  />

  <author>

    <name>Tên kênh</name>

    <uri>Link dẫn đến kênh</uri>

  </author>

  <published>Thời gian kênh được lập</published>
```

```

<entry>

  <id>yt:video:id_of_video</id>

  <yt:videoId>id_of_video</yt:videoId>

  <yt:channelId>ID_OF_CHANNEL</yt:channelId>

  <title>

    Tên của video

  </title>

  <link rel="alternate" href="link đến video" />

  <author>

    <name>Tên kênh</name>

    <uri>Link đến kênh</uri>

  </author>

  <published>Thời gian video được xuất bản</published>

  <updated>Thời gian gần nhất video được sửa</updated>

  <media:group>

    <media:title

      >Tên video</media:title

    >

    <media:content

      url="url của video"

      type="type của video"

      width="640"

      height="390"

    />

    <media:thumbnail

```

```

        url="link thumbnail

        width="480"

        height="360"

    />

    <media:description

        >Mô tả về video</media:description

    >

    <media:community>

        <media:starRating count="1343" average="5.00"
min="1" max="5" />

        <media:statistics views="14437" />

    </media:community>

</media:group>

</entry>

<entry>

    ...

</entry>

...

</feed>

```

c. Thuật toán chuyển đổi XML thành JSON

Vì dự án chạy trên nền tảng web, sử dụng ngôn ngữ lập trình Javascript. Hơn nữa không phải tất cả những thông tin của file XML trên cũng cần được sử dụng, vì vậy cần phải chuyển đổi nội dung XML trên trở lại thành định dạng JSON chứa những dữ liệu cần thiết để có thể xử lý được.

Bước đầu tiên cần làm là phải lấy được nội dung theo định dạng XML. Khi yêu cầu với link API trên, ta nhận là một trang web có nội dung body chứa thông tin XML đã trình bày, chứ ta không nhận lại một đối tượng XML có thể xử lý được. Để có thể lấy được đối tượng XML có thể xử lý, bước đầu tiên ta lấy nội dung

của trang web trả về dưới dạng chuỗi (String), sau đó bước hai sẽ tiến hành chuyển đổi chuỗi đó sang XML. Mã nguồn bước 1 như sau:

```
fetch(rssYoutubeAPI + ID, responseOption)

.then((response) => {

    data = response.text();

    return data;

});
```

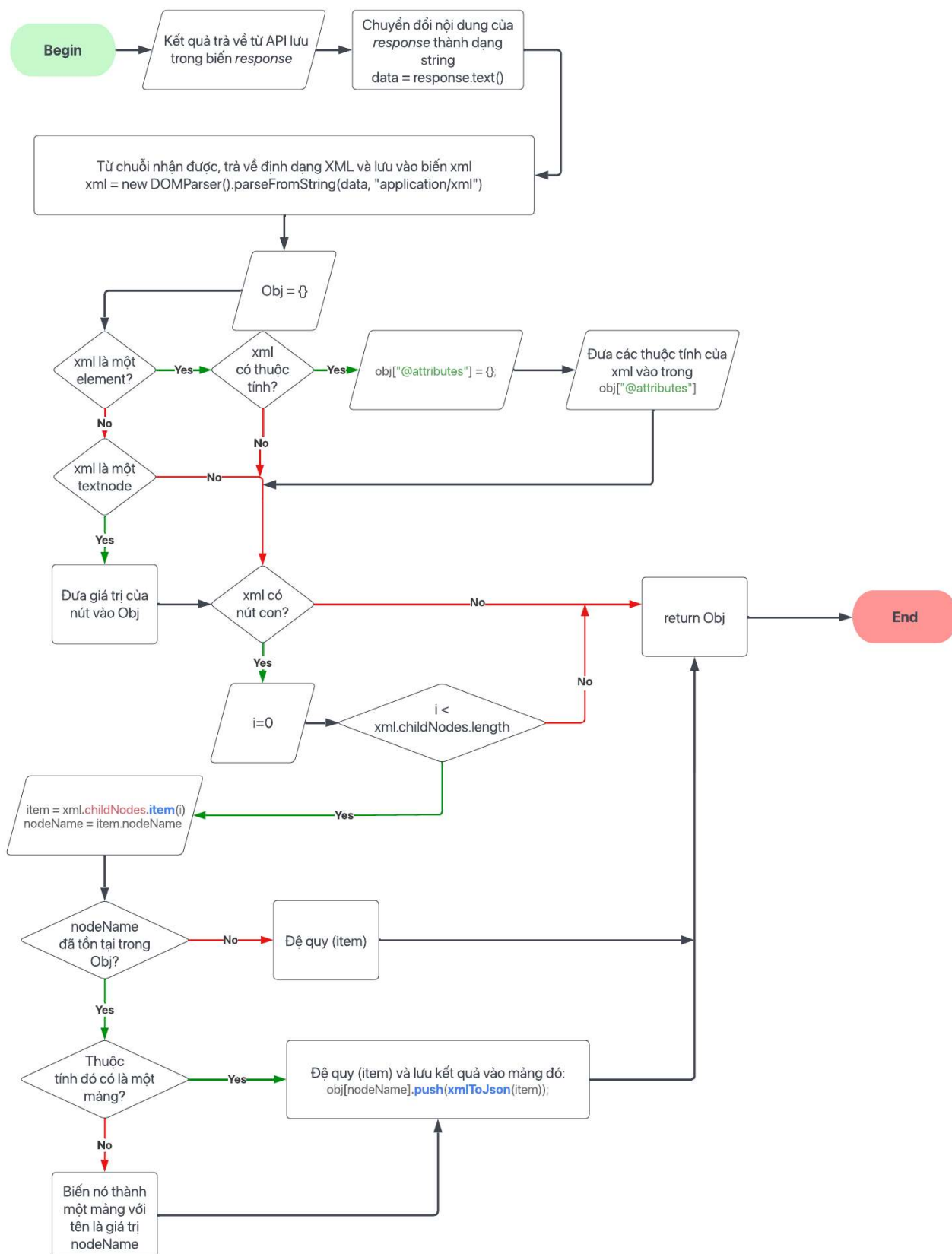
Như vậy ta đã có biến *data* là một chuỗi (string) mã XML chúng ta lấy được từ API. Ở bước hai, ta sẽ biến nó thành một định dạng nào đó có thể đọc được từ thẻ và thuộc tính của thẻ. Để làm được điều đó, ta sẽ sử dụng phương thức *DOMParser()* và *parseFromString()*. Hai phương thức này sẽ phân tích cú pháp của chuỗi *data* biến nó trở thành một mô hình dạng cây giống như DOM (Document Object Model) với các nút trên cây có thể là một thẻ (element), thuộc tính (attributes) hay nội dung (text). Vì *data* chứa nội dung của một file XML nên giá trị trả về từ hai phương thức này cũng phải thuộc dạng XML. Ta có đoạn mã như sau:

```
const parser = new DOMParser();

const xlm = parser.parseFromString(data,
    "application/xml");
```

Với đối số thứ hai là chuỗi *application/xml* ta đã có được giá trị trả về của phương thức *parseFromString()* là một mô hình dạng cây mô phỏng một định dạng XML. Biến *xlm* là con trỏ trỏ đến nút gốc của mô hình cây đó.

Ta tiến hành viết hàm đệ quy để trả về một đối tượng Javascript theo dạng JSON. Thuật toán được biểu diễn thông qua sơ đồ sau:



Hình 25. Thuật toán đệ quy chuyển đổi XML thành Javascript Object

Mã nguồn hàm chuyển đổi XML thành JavaScript:

```
function xmlToJson(xml) {  
  
    // Khởi tạo biến object để lưu giá trị trả về  
  
    var obj = {};  
  
    //Kiểm tra gốc  
  
    if (xml.nodeType === 1) {  
  
        //gốc là một element hay thẻ xml  
  
        // Kiểm tra các nút thuộc tính của gốc  
  
        //Thêm giá trị các nút thuộc tính vào thuộc tính  
        "@attributes" của đối tượng  
  
        if (xml.attributes.length > 0) {  
  
            obj["@attributes"] = {};  
  
            for (var j = 0; j < xml.attributes.length; j++) {  
  
                var attribute = xml.attributes.item(j);  
  
                obj["@attributes"][attribute.nodeName] =  
                attribute.nodeValue;  
  
            }  
  
        }  
  
        } else if (xml.nodeType === 3) {  
  
            //gốc là một text  
  
            obj = xml.nodeValue; //Trả lại giá trị của gốc  
  
        }  
  
        // Nếu gốc có nút con
```

```

    if (xml.hasChildNodes()) {

        //Xử lý từng nút con một

        for (var i = 0; i < xml.childNodes.length; i++) {

            var item = xml.childNodes.item(i);

            var nodeName = item.nodeName; //Lưu tên của nút
con

            //Kiểm tra tên nút con đó đã tồn tại như một
thuộc tính của object chưa

            if (typeof obj[nodeName] == "undefined") {

                //Nếu chưa, đệ quy hàm với nút con đó là gốc

                obj[nodeName] = xmlToJson(item);

            } else {

                //Nếu rồi, khả năng nào nó là một mảng.

                //Kiểm tra thuộc tính đó là một mảng hay không.

                if (typeof obj[nodeName].push == "undefined") {

                    //Nếu không phải một mảng, biến nó thành một
mảng

                    var old = obj[nodeName];

                    obj[nodeName] = [];

                    obj[nodeName].push(old);

                }

                /* Nếu nó là một mảng

                => Chạy đệ quy hàm với nút con hiện tại là gốc

                => Lưu kết quả vào mảng trên */

                obj[nodeName].push(xmlToJson(item));

            }

```

```

    }

    }

    return obj; //trả về giá trị obj cho hàm

}

```

IV. Mã nguồn chương trình và demo:

a. Mã nguồn Client:

Client sử dụng thư viện ReactJS, dưới đây là mã nguồn tập tin gốc App.js:

```

import './App.css';

import Navigate from './components/Navigate/Navigate';

import Content from './components/Content/Content';

import Add from './components/Add&Delete/Add.';

import { useEffect, useState } from 'react';

import io from 'socket.io-client';

//Tạo socket kết nối đến ip của server

const socket = io.connect("http://192.168.183.133:3001/");

//component App

function App() {

    //Tạo mảng các kênh theo dõi. Tạo giá trị mặc định

    const [allchannels, setAllChannel] = useState([

        {

            channelId: "#default",

            title: "#default",

```

```
    },  
  ]);
```

//Hàm Lấy danh sách các kênh theo dõi từ server

```
const getFromServer = (bool) => {  
  socket.emit("loadPage", { message: "loaded" });  
  socket.on("store", (data) => {  
    setAllChannel((prev) => {  
      console.log(data);  
      return data;  
    });  
    if (bool) window.location.reload(true);  
  });  
};
```

//Mỗi khi component reload sẽ lấy lại thông tin từ server

```
useEffect(() => {  
  getFromServer(false);  
}, []);
```

```
useEffect(() => {
```

//Tạo sự kiện click cho nút thêm kênh

```
  const menu_addChannel =  
  document.getElementById("addMenu#addChannel");  
  menu_addChannel.addEventListener("click", () => {
```

```

        setIsAdding(true);
    });

    return () => {
        menu_addChannel.removeEventListener("click", () => {
            setIsAdding(true);
        });
    };
}, []);

//Biến Lưu ID của kênh muốn xem nội dung
//Mặc định biến này là kênh đầu tiên của danh sách

const [contentID, setContentID] =
useState(allchannels[0].channelId);

//Biến xác định chức năng thêm kênh
//Mặc định là không

const [isAdding, setIsAdding] = useState(false);

//Hàm set lại id của kênh cần xem nội dung

const reloadID = (id) => {
    setContentID(id);
};

//Hàm xóa kênh

const deleteChannel = (id) => {

```

```

console.log("deleting: " + id);

//Emit sự kiện xóa kênh đến server

//Gửi kèm id của kênh cần xóa

socket.emit("delete_channel", id);

//Nhận Lại sự kiện đã xóa từ server

//Reload lại trang để lấy lại danh sách các kênh

socket.on("delete_done", (mess) => {

    console.log(mess);

    setContentID(allchannels[0].channelId);

    getFromServer(true);

});

};

//Hàm thêm kênh

const addChannel = (id, title) => {

    const newChannel = {

        channelId: id,

        title: title,

    };

    //Emit sự kiện thêm kênh đến server

    //Gửi kèm thông tin của kênh mới

    socket.emit("add_channel", newChannel);

    //Nhận Lại sự kiện đã thêm kênh từ server

    //Reload lại trang để lấy lại danh sách kênh

```

```
socket.on("add_done", (data) => {  
    console.log(data);  
    getFromServer(true);  
});  
};
```

//callback lấy id của kênh cần xem thông tin

```
const getContentID = () => {  
    return contentID;  
};
```

//callback set trạng thái đang thêm kênh hay không

```
const setIsAddingCallback = (bool) => {  
    setIsAdding(bool);  
};
```

//Mã HTML trả về

```
return (  
    <div className="App">  
        <div className="layOut">
```

```

<div className="header">

  <div id="logo"></div>

  <div id="webName">RSTube</div>

</div>

<div className="boards">

  <Navigate all={allchannels} reloadID={reloadID} />

</div>

{allchannels[0].channelId !== "#default" && (

  <div className="articles">

    {!isAdding ? (

      <Content

        getContentID={getContentID()}

        deleteChannel={deleteChannel}

      />

    ) : (

      <Add

        callback={setIsAddingCallback}

        addCallback={addChannel}

        allchannels={allchannels}

      />

    )}

  </div>

)}

```



```

        </div>

    </div>

    );
}

```

```
export default App;
```

b. Mã nguồn Server

```

const express = require("express");

const app = express();

const http = require("http");

const { Server } = require("socket.io");

const cors = require("cors");

const mongoose = require("mongoose");

const mongoDB =

    "mongodb+srv://TriTamLe:Nhuanthien@cluster0.y1jlcoj.mongodb
    .net/?retryWrites=true&w=majority";

    //Thiết lập kết nối trên cơ sở dữ liệu MongoDB

mongoose.set("strictQuery", false);

mongoose

    .connect(mongoDB, {

        useNewUrlParser: true,

        useUnifiedTopology: true,

    })

```

```

    .then(() => {
        console.log("connected to Database");
    })

    .catch((err) => {
        console.log("Error");
        console.log(err);
    });

//Khởi tạo model cho đối tượng kênh

const Schema = mongoose.Schema;

const ChannelSchema = new Schema(
    {
        channelId: String,
        title: String,
    },
    {
        collection: "channels",
    }
);

const ChannelModel = mongoose.model("channel",
ChannelSchema);

//Khởi tạo biến allchannels quản lý danh sách kênh trên
database

let allchannels;

//Thiết lập cors để giới hạn các ip được phép kết nối
//Ở đây ta mặc định cho mọi ip đều có thể

app.use(cors());

```

```

const server = http.createServer(app);

const io = new Server(server, {
  cors: {
    origin: "0.0.0.0:3000",
    methods: ["GET", "POST"],
  },
});

//Thiết lập kết nối với socket gửi yêu cầu
io.on("connect", (socket) => {
  //Nhận sự kiện load trang từ Client
  socket.on("loadPage", (data) => {
    console.log(data.message);

    //Lấy danh sách tất cả các kênh từ database
    ChannelModel.find({}).then((data) => {
      allchannels = data;
    });

    //Emit sự kiện trả lại danh sách kênh cho client
    socket.emit("store", allchannels);
  });

  //Nhận sự kiện thêm kênh
  socket.on("add_channel", (data) => {
    ChannelModel.create(data)
      .then(() => {
        console.log("added", data);
      });
  });
});

```

```

    })

    .then(() => {

        socket.emit("add_done", "done");

    })

    .catch((err) => {

        console.log("ERROR", err);

    });

});

//Nhận sự kiện xóa kênh

socket.on("delete_channel", (id) => {

    console.log(id);

    console.log("deleting");

    ChannelModel.find({

        channelId: id,

    })

    .then((data) => {

        ChannelModel.deleteOne(data[0]).then((done) => {

            socket.emit("delete_done", "done");

        });

    })

    .catch((err) => {

        console.log("ERROR", err);

    });

});

```

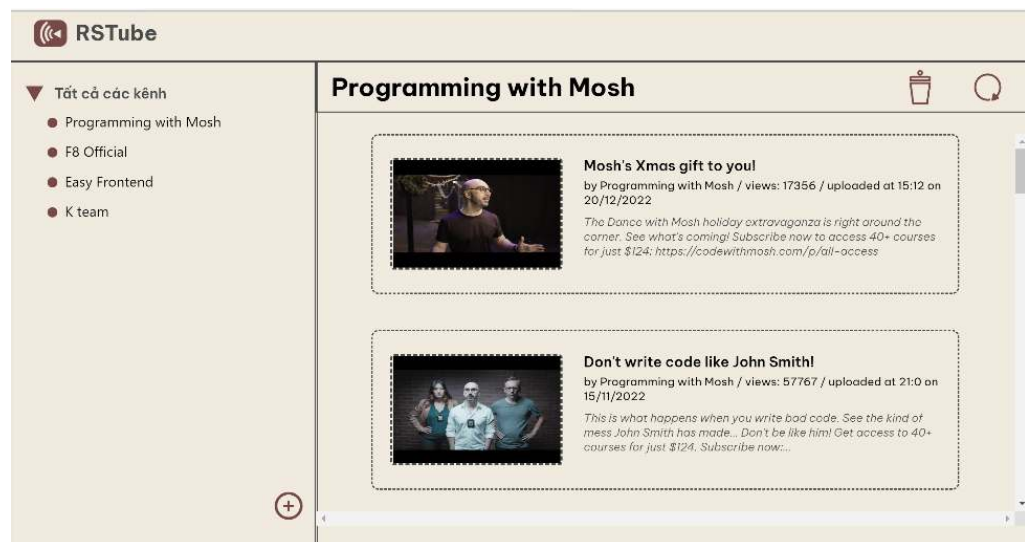
```
});
```

```
//Chạy server ở cổng 3001
```

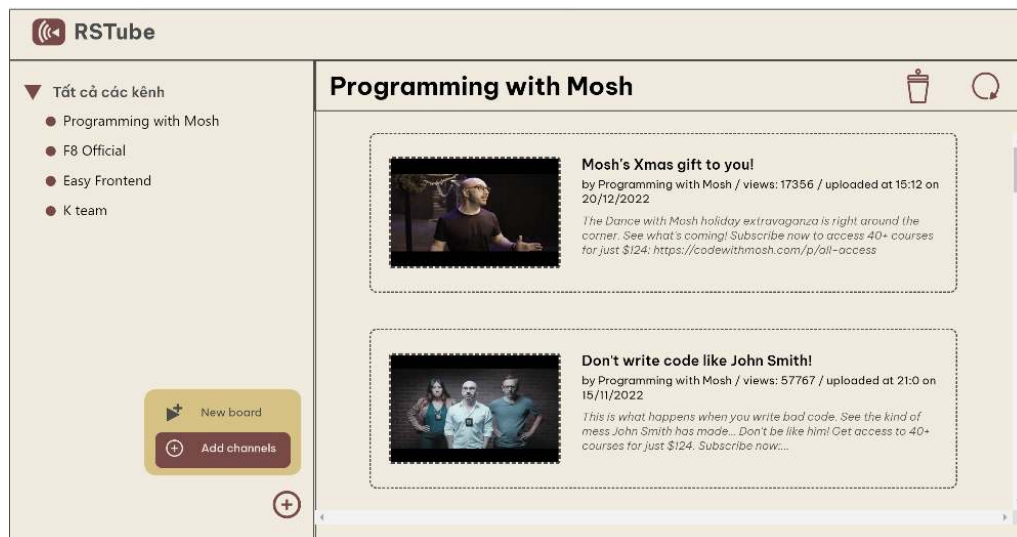
```
server.listen(3001, () => {  
  console.log("Server is running");  
});
```

c. Hình ảnh demo:

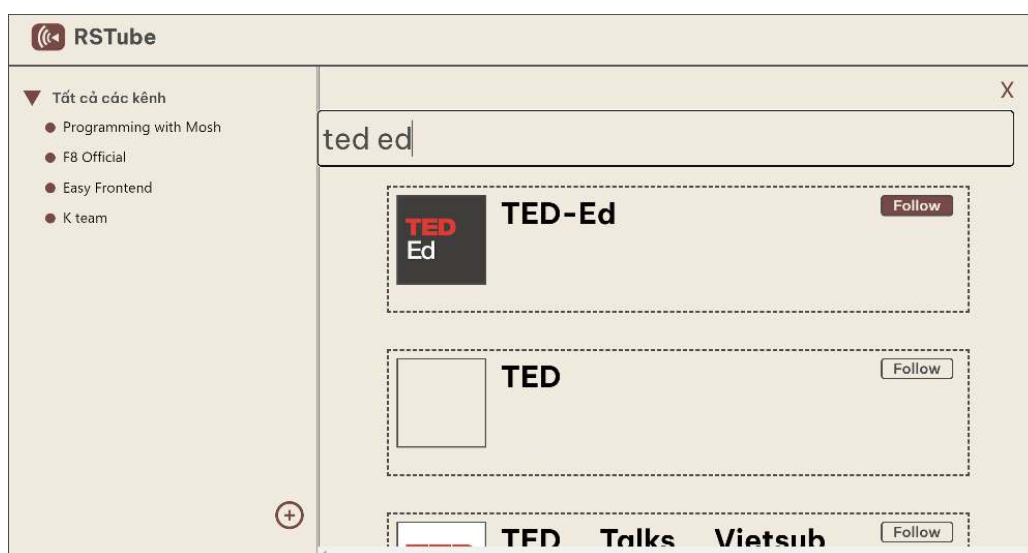
Các hình ảnh giao diện của Client:



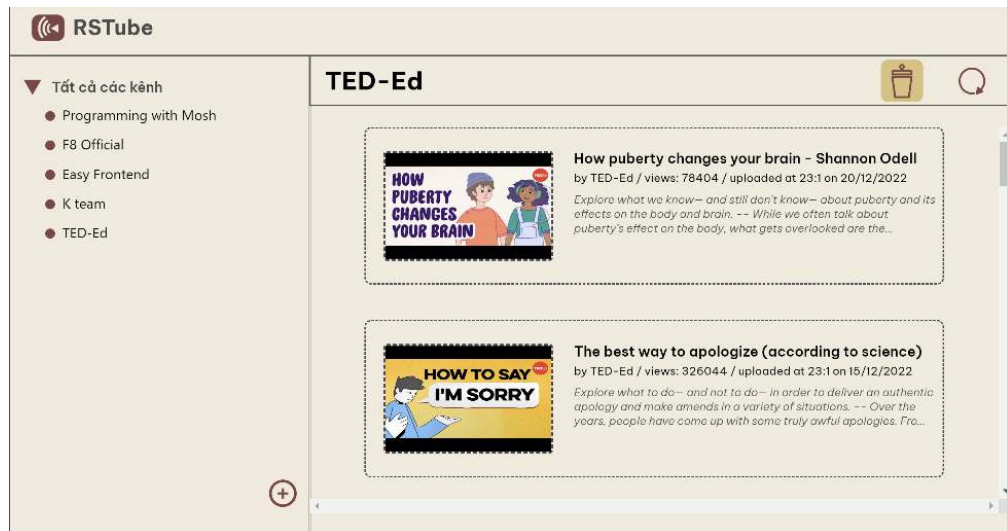
Hình 26. Giao diện tổng quan trang web



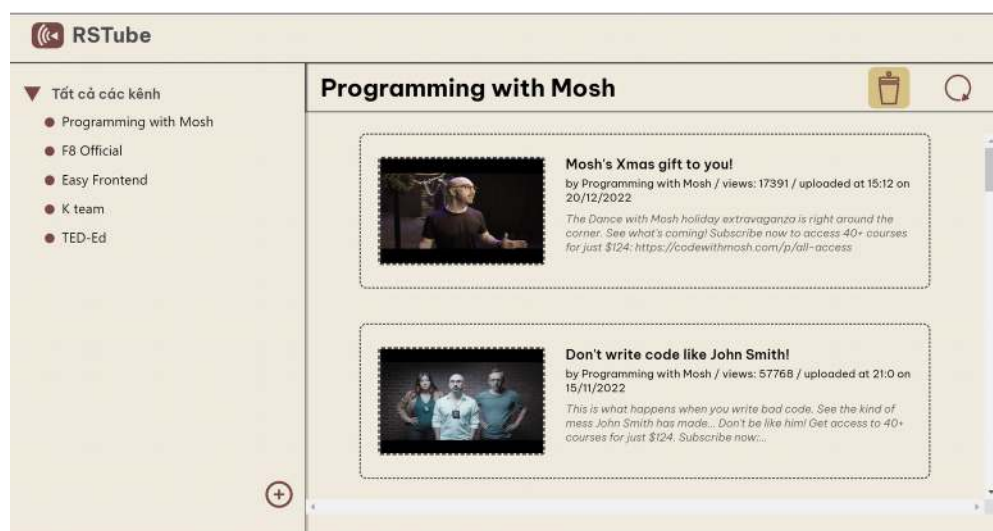
Hình 27. Chức năng thêm kênh mới



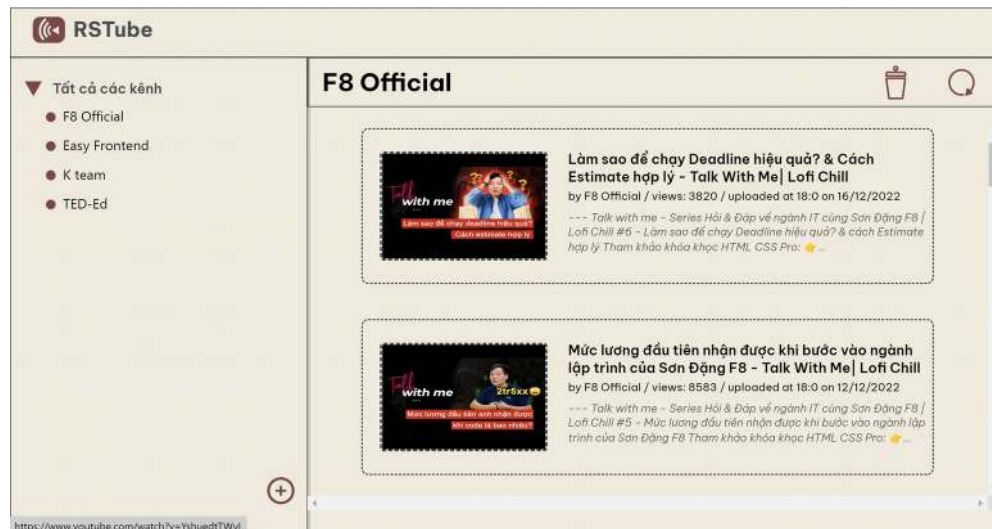
Hình 28. Tìm kiếm và thêm kênh Ted-Ed



Hình 29. Giao diện thông tin kênh mới thêm vào

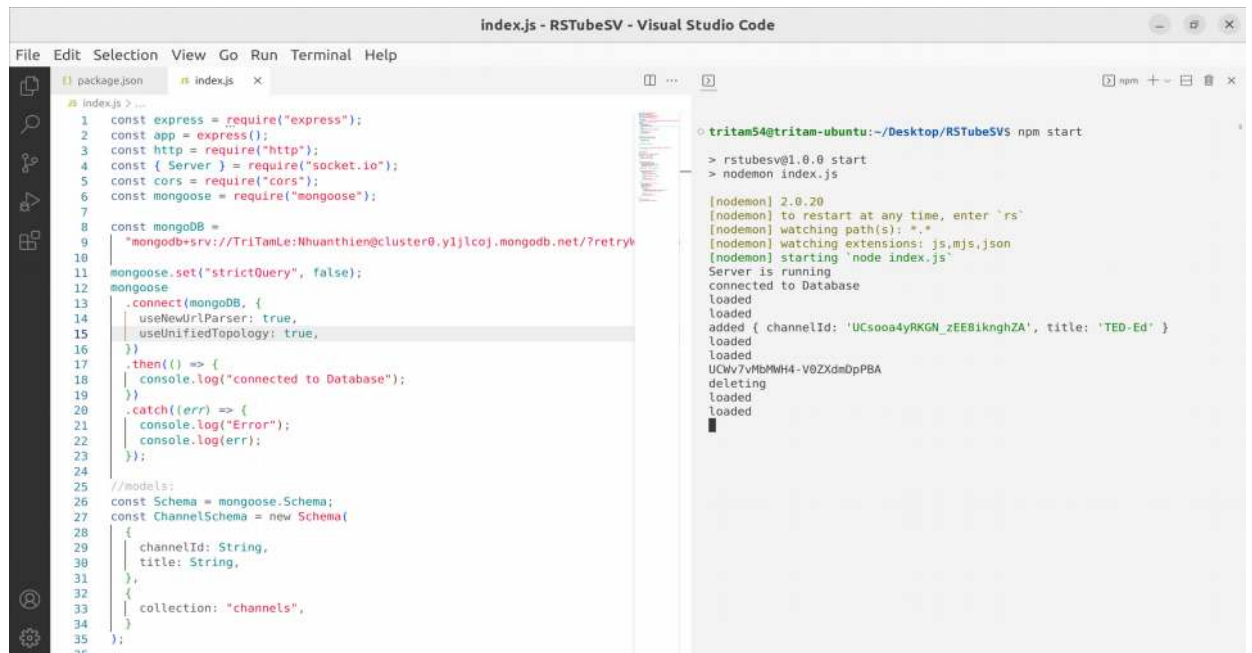


Hình 30. Tiến hành xóa kênh



Hình 31. Giao diện sau khi xóa

Hình ảnh các kết quả trả về của server trên cosole:



Hình 32. Nội dung console của Server khi thao tác các bước trên client

PHỤ LỤC VÀ TÀI LIỆU THAM KHẢO

a. Phần hệ điều hành:

Andrew S. Tanenbaum - Modern Operating Systems

Tiểu luận: liên lạc giữa hai tiến trình sử dụng pipe của Đại học Công nghiệp thành phố Hồ Chí Minh

b. Phần mạng:

<https://www.npmjs.com/package/youtube-search-api>

<https://developer.mozilla.org/en-US/docs/Web/API/DOMParser>

https://www.w3schools.com/xml/dom_nodetype.asp

[https://learn.microsoft.com/en-us/previous-versions/windows/desktop/ms753745\(v=vs.85\)](https://learn.microsoft.com/en-us/previous-versions/windows/desktop/ms753745(v=vs.85))

<https://davidwalsh.name/convert-xml-json>